

TechStore Plus

A Jornada de um Atendente Virtual Inteligente

Explicação completa do projeto — semanas 1 a 9
escrita para quem não é da área de programação

Bruno Conte Pieri

Programa DevOps / ML Serving — Pluralit

Julho de 2026

Sumário

Introdução — a história, como ler, conceitos fundamentais

PARTE I — Módulo 1: O Nascimento do Atendente

Capítulo 1 · Semana 1 — Conversando com a IA do jeito mais direto

Capítulo 2 · Semana 2 — A linha de montagem (LangChain LCEL)

Capítulo 3 · Semana 3 — Memória de longo prazo e mãos para trabalhar

Capítulo 4 · Desafio M1 — Do laboratório para o ar

PARTE II — Módulo 2: O Atendente que Estuda (RAG)

Capítulo 5 · Semana 4 — Ensinando o atendente a consultar documentos

Capítulo 6 · Semana 5 — Afinando a biblioteca (otimização do RAG)

Capítulo 7 · Semana 6 — A biblioteca à prova de produção (desafio M2)

PARTE III — Módulo 3: O Atendente que Raciocina em Etapas

Capítulo 8 · Semana 7 — Abrindo a caixa-preta: o fluxo desenhado à mão

Capítulo 9 · Semana 8 — Do agente único à central de especialistas

Capítulo 10 · Semana 9 — A equipe supervisionada (desafio M3)

Conclusão — a linha do tempo e os cinco princípios

Glossário — todos os termos técnicos, sem jargão

Apêndice — mapa de repositórios e arquivos

TechStore Plus — A Jornada de um Atendente Virtual

Explicação completa do projeto, semanas 1 a 9, para quem não é da área

Introdução

A história que este documento conta

Imagine uma loja de eletrônicos chamada **TechStore Plus**. Ela vende celulares, notebooks, fones de ouvido — e, como toda loja, recebe dezenas de mensagens de clientes por dia: *"meu pedido não chegou"*, *"como devolvo esse produto?"*, *"qual notebook você recomenda para estudar engenharia?"*.

Este projeto construiu, ao longo de nove semanas, um **atendente virtual** (um *chatbot*) capaz de responder essas mensagens. Mas não de qualquer jeito: a cada semana o atendente ganhou uma habilidade nova, como um funcionário que vai sendo treinado e promovido.

A evolução aconteceu em **três módulos**, cada um com três semanas e um desafio prático no final:

Módulo	Semanas	O que o atendente aprendeu	Desafio
1 — O nascimento	1 a 3	Conversar, entender o cliente, lembrar do que foi dito e usar ferramentas	Ir para produção: virar um site de verdade, no ar
2 — O estudo	4 a 6	Consultar documentos da loja antes de responder (a técnica chamada RAG)	Um sistema de consulta robusto, com trava contra respostas inventadas
3 — O raciocínio	7 a 9	Trabalhar em etapas planejadas, delegar para especialistas e desfazer erros	Uma equipe de atendentes coordenada por um supervisor

Cada capítulo deste documento explica uma dessas semanas: **o que** foi construído, **por que** foi construído daquele jeito, e **como** o código funciona — sempre em linguagem comum, com analogias do dia a dia, e com trechos de código comentados linha por linha para quem quiser espiar por baixo do capô.

Como ler este documento

Você **não precisa saber programar** para acompanhar. O documento foi organizado para três tipos de leitura:

- Leitura corrida** — leia só o texto e as analogias, pulando os quadros de código. Você vai entender o que o sistema faz e por quê.
- Leitura curiosa** — leia também os quadros "Por dentro do código". Cada trecho vem com uma tradução em português do que cada linha faz.
- Leitura de referência** — use o glossário no final sempre que um termo técnico aparecer. Termos do glossário aparecem em **negrito** na primeira vez que surgem em cada capítulo.

Os quadros seguem um padrão:



Analogia — uma comparação com o mundo real para fixar a ideia.

Os blocos como este são código de verdade do projeto,
sempre acompanhados de comentários explicando cada parte.

Conceitos Fundamentais — o mínimo que você precisa saber

Antes da semana 1, vale um mini-curso de dez minutos. São sete conceitos que aparecem no projeto inteiro. Se você já conhece, pule para o próximo capítulo.

1. O que é um programa de computador?

Um programa é uma **receita de bolo escrita para o computador**: uma lista de instruções, em ordem, que a máquina segue ao pé da letra. A diferença para uma receita de verdade é que o computador não improvisa — se a instrução estiver ambígua ou errada, ele erra junto, sem perceber.

2. O que é Python?

Python é a linguagem em que este projeto foi escrito — uma das "línguas" que humanos usam para escrever instruções que o computador entende. Foi escolhida por ser a mais popular no mundo da inteligência artificial e por ser relativamente legível: um código Python bem escrito quase se lê como inglês.

```
# Exemplo do mundo real: três linhas de Python
nome = "Maria"                # guarda o texto "Maria" numa caixinha chamada 'nome'
saudacao = f"Olá, {nome}!"    # monta a frase usando o conteúdo da caixinha
print(saudacao)               # mostra na tela: Olá, Maria!
```

As "caixinhas" (`nome` , `saudacao`) se chamam **variáveis**. Blocos de instruções reutilizáveis se chamam **funções** — pense numa função como um eletrodoméstico: você coloca algo dentro (os *parâmetros*), aperta o botão, e sai um resultado.

3. O que é um LLM (o "cérebro" do chatbot)?

LLM significa *Large Language Model* — "grande modelo de linguagem". É um programa treinado com uma quantidade gigantesca de textos, que aprendeu a **prever qual palavra vem a seguir** em qualquer frase. Dessa habilidade aparentemente simples emerge algo poderoso: ele consegue conversar, resumir, traduzir e responder perguntas.

O LLM usado neste projeto se chama **GPT-4o-mini**, da empresa OpenAI (a mesma do ChatGPT). Ele não roda no computador do projeto — roda nos servidores da OpenAI, e o nosso código conversa com ele pela internet.

💡 **Analogia** — o LLM é como um consultor extremamente culto que atende por telefone. Ele leu praticamente tudo que existe, mas: (1) você paga por minuto de conversa, (2) ele não conhece a *sua* empresa, e (3) de vez em quando ele responde com confiança algo que está errado. Boa parte deste projeto é aprender a trabalhar com esse consultor: dar contexto a ele, conferir as respostas e não gastar telefonema à toa.

4. O que é uma API?

API (*Application Programming Interface*) é a forma como dois programas conversam entre si. Se um site tem um balcão de atendimento para pessoas (botões, telas), a API é o **balcão de atendimento para outros programas**: um endereço na internet onde um programa faz um pedido padronizado e recebe uma resposta padronizada.

Quando nosso chatbot precisa do LLM, ele faz uma **chamada de API** para a OpenAI: envia a pergunta do cliente e recebe a resposta gerada. Cada chamada custa alguns centavos — por isso o projeto se preocupa tanto em fazer menos chamadas e chamadas mais inteligentes.

A **chave de API** (*API key*) é a senha que identifica quem está pedindo — como um cartão corporativo: quem tem o cartão pode usar o serviço, e a conta chega no dono do cartão. Por isso ela nunca é publicada junto com o código.

5. O que é um prompt?

Prompt é o texto de instruções que enviamos ao LLM. É literalmente o que você "fala" para o consultor do telefone antes de fazer a pergunta. Há dois tipos importantes no projeto:

- **Prompt de sistema** (*system prompt*): as instruções permanentes — "você é um atendente da TechStore Plus, seja educado, siga estes passos...". Define a personalidade e as regras.
- **Mensagem do usuário**: a pergunta do cliente naquele momento.

Grande parte da "programação" de um chatbot moderno é, na verdade, **escrever bons prompts** — instruções claras, com exemplos, que não deixem margem para o LLM inventar.

6. O que é JSON?

JSON é um formato de texto para organizar informações de um jeito que tanto humanos quanto programas conseguem ler. Pense numa **ficha cadastral padronizada**:

```
{
  "cliente": "Maria Silva",
  "categoria": "devolução",
  "urgencia": "alta",
  "produtos_mencionados": ["iPhone 15"]
}
```

O projeto usa JSON o tempo todo: para guardar o histórico das conversas em arquivos, e para obrigar o LLM a responder de forma estruturada (em vez de um texto solto, uma ficha preenchida — que o programa consegue processar automaticamente).

7. O que são bibliotecas e frameworks?

Ninguém constrói uma casa fabricando os próprios tijolos. Em programação, **bibliotecas** são caixas de peças prontas feitas por outras pessoas, que o projeto importa e usa. As principais aqui:

Biblioteca	O que faz no projeto
OpenAI	Conversa com o LLM GPT-4o-mini
LangChain	Organiza as etapas do chatbot como uma linha de montagem (semana 2 em diante)
LangGraph	Permite montar fluxos com decisões, desvios e memória (semanas 7-9)
Pydantic	Confere se os dados têm o formato certo — um "fiscal de qualidade" de fichas
ChromaDB	Banco de dados especial que busca por <i>significado</i> , não por palavra exata (semanas 4-6)
FastAPI	Cria o "balcão de atendimento" (API) do nosso próprio sistema (desafio M1)
pytest	Roda testes automáticos que conferem se tudo continua funcionando

Um **framework** é uma biblioteca grande que dita a estrutura do programa — você preenche as lacunas dele, e não o contrário.

Um mapa mental para levar para o resto do documento

Tudo que vem pela frente é variação de um mesmo ciclo:

```
mensagem do cliente
  ↓
[preparar contexto]    ← o que o atendente precisa saber para responder bem?
  ↓
[chamar o LLM]         ← o telefonema para o consultor
  ↓
[conferir e estruturar] ← a resposta está no formato certo? é confiável?
  ↓
[agir e registrar]      ← responder o cliente, salvar histórico, acionar ferramentas
```


- O **módulo 1** constrói esse ciclo e o coloca no ar.
- O **módulo 2** turбина a etapa "preparar contexto" com documentos da loja (RAG).
- O **módulo 3** transforma o ciclo único em um **fluxo com várias etapas e vários especialistas**.

Boa leitura!

PARTE I — Módulo 1: O Nascimento do Atendente

Capítulo 1 · Semana 1 — Conversando com a IA do jeito mais direto

Arquivo: `TechStorePlus_Customer_Service_Chatbot_Project.ipynb`

O que foi construído

Na primeira semana nasceu a versão mais simples possível do atendente: um programa que recebe a mensagem do cliente, liga para o "consultor" (o LLM **GPT-4o-mini**, da OpenAI), e devolve a resposta. Sem atalhos, sem frameworks — cada passo escrito à mão, justamente para entender o que acontece por baixo.

Mesmo simples, essa versão já faz cinco coisas que um atendente de verdade faria:

1. **Entende e classifica** a mensagem: é problema técnico? cobrança? devolução? Qual a urgência? O cliente está irritado?
2. **Responde** de forma personalizada, seguindo as políticas da loja.
3. **Lembra da conversa** enquanto ela dura (se o cliente disse o nome na primeira mensagem, o bot não pergunta de novo na terceira).
4. **Resume e arquiva** cada conversa num arquivo padronizado.
5. **Encaminha** o caso para o time certo (suporte técnico, faturamento, devoluções...).

 **Analogia** — pense num atendente de balcão no primeiro dia de trabalho, com um manual da empresa na mão. Ele ainda não conhece os sistemas internos nem o estoque, mas já sabe: ouvir, entender o tipo de problema, responder com educação seguindo o manual, anotar tudo numa ficha e encaminhar para o setor certo.

As peças, uma a uma

1. O manual da empresa (`COMPANY_CONTEXT`)

Lembra que o LLM é um consultor culto que **não conhece a sua empresa**? A primeira peça resolve isso: uma "ficha da empresa" com tudo que o atendente precisa saber — nome, horários, produtos, políticas de troca e frete.

```
# Ficha da empresa: um dicionário Python (pares "rótulo: valor")
COMPANY_CONTEXT = {
    "company_name": "TechStore Plus",
    "business_hours": {
        "monday_friday": "09:00-18:00",    # seg-sex, 9h às 18h
        "saturday": "10:00-14:00"         # sábado, 10h às 14h
    },
    "products": ["Laptops", "Smartphones", "Tablets", "Headphones", ...],
    "policies": {
        "shipping": "Free nationwide shipping for purchases over $500.",
        "returns": "30 days for exchanges and 7 days for refunds.",
        # ... garantia de 12 meses, financiamento etc.
    },
}
```

Tradução: isso é só uma estrutura de dados — uma ficha organizada. O pulo do gato é o que fazemos com ela a seguir.

2. As instruções permanentes (`SYSTEM_ROLE`)

Aqui a ficha da empresa é **injetada dentro do prompt de sistema** — as instruções permanentes que o LLM recebe antes de qualquer conversa:

```
SYSTEM_ROLE = f"""
You are a professional customer service assistant for {COMPANY_CONTEXT['company_name']}.

Your behavior:
- Be polite, helpful, clear, and professional.           # seja educado e profissional
- If the customer is upset, acknowledge with empathy.    # se o cliente estiver chateado, demonstre
empatia
- Do not invent unavailable company policies.             # NÃO invente políticas que não existem
- Use the company context below as your source of truth.  # use a ficha abaixo como fonte da verdade

Company context:
{json.dumps(COMPANY_CONTEXT, indent=2)}    # ← a ficha inteira entra aqui, em formato JSON

Conversation flow:
1. Personalized greeting.                               # 1. cumprimente
2. Collect basic customer information...                  # 2. colete nome, e-mail, nº do pedido
3. Identify inquiry type...                              # 3. identifique o tipo de dúvida
4. Route the case...                                     # 4. encaminhe para o time certo
5. Give clear next steps.                                # 5. dê os próximos passos
"""
```

Tradução: o `f"""..."""` é um texto com lacunas — o Python preenche `{...}` com valores reais. Estamos literalmente escrevendo o **roteiro de treinamento do atendente**, incluindo a regra de ouro dos chatbots corporativos: *"não invente políticas"*. Sem essa instrução, o LLM tenderia a responder qualquer pergunta com confiança — mesmo inventando prazos de devolução que não existem.

3. A memória da conversa (`ConversationSession`)

Um detalhe surpreendente sobre LLMs: **eles não lembram de nada entre uma chamada e outra**. Cada telefonema para o consultor começa do zero. Quem lembra da conversa é o *nosso* programa — e ele reenvia o histórico inteiro a cada nova mensagem.

```
class ConversationSession:
    """Guarda o histórico de conversa de um cliente."""

    def __init__(self, customer_id=None):
        # Cada cliente ganha um código único, tipo protocolo: CUST-78C546C3
        self.customer_id = customer_id or f"CUST-{uuid.uuid4().hex[:8].upper()}"
        # A lista de mensagens já nasce com as instruções permanentes dentro
        self.messages = [{"role": "system", "content": SYSTEM_ROLE}]

    def add_user_message(self, message):      # anota o que o cliente disse
        self.messages.append({"role": "user", "content": message})

    def add_assistant_message(self, message): # anota o que o bot respondeu
        self.messages.append({"role": "assistant", "content": message})
```

Tradução: uma `class` é um molde para criar objetos — aqui, o molde de uma "pasta de atendimento". Cada cliente novo ganha sua pasta com número de protocolo, e cada fala (do cliente ou do bot) vai sendo anexada nela. Quando o bot precisa responder, ele manda **a pasta inteira** para o LLM — é assim que o "consultor sem memória" parece lembrar da conversa.

 **Analogia** — é como um call center em que cada ligação cai num atendente diferente, mas todos têm acesso à ficha completa das ligações anteriores. O atendente individual não lembra de você; a *ficha* lembra.

4. O classificador (`analyze_customer_query`)

Antes de responder, o sistema faz uma **primeira chamada ao LLM só para entender a mensagem** — e exige a resposta em formato de ficha (JSON), não texto livre:

```
def analyze_customer_query(query):
    prompt = f"""
    Analyze the following customer service query.

    Return ONLY valid JSON with this exact structure: # devolva SOMENTE JSON, neste formato exato:
    {{
      "customer_sentiment": "positive | neutral | negative", # sentimento do cliente
      "query_category": "technical | billing | return | ...", # categoria (8 opções)
      "urgency_level": "low | medium | high", # urgência
      "mentioned_products": ["product names if any"], # produtos citados
      "recommended_routing": "team or department name", # para qual time encaminhar
      "reasoning_summary": "short explanation" # justificativa curta
    }}

    Customer query:
    {query} # ← a mensagem real do cliente entra aqui
    """

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0, # temperatura 0 = modo "burocrata": máxima consistência, zero
        # criatividade
        messages=[...]
    )
    content = response.choices[0].message.content
    try:
        return json.loads(content) # tenta ler a ficha JSON
    except json.JSONDecodeError:
        # Plano B: o LLM às vezes devolve a ficha embrulhada em ```json ... ```
        cleaned = content.strip().replace("```json", "").replace("```", "").strip()
        return json.loads(cleaned)
```

Dois detalhes importantes aqui:

- `temperature=0` — a "temperatura" controla a criatividade do LLM. Zero significa: responda sempre do jeito mais previsível. Para classificar, queremos um burocrata, não um poeta.
- **O plano B do `try/except`** — o LLM às vezes desobedece e devolve o JSON embrulhado em formatação extra. O código limpa manualmente. *Guarde esse incômodo*: a semana 2 existe em grande parte para eliminar essa gambiarra.

5. O redator (`generate_personalized_response`)

Com a análise em mãos, uma **segunda chamada ao LLM** gera a resposta ao cliente — desta vez com `temperature=0.4` (um pouco de naturalidade na escrita) e com instruções que usam a análise:

```

response_instruction = f"""
Customer query analysis:
{json.dumps(analysis, indent=2)}          # ← a ficha da análise entra aqui

Generate a professional customer service response.
Requirements:
- If sentiment is negative, start with empathy.      # cliente irritado? comece com empatia
- If urgency is high, acknowledge priority...        # urgente? reconheça e dê passos imediatos
- Include specific information from company policy.  # cite a política real da loja
- End with the next best action.                    # termine com o próximo passo
"""

```

Tradução: o sistema usa a classificação para **calibrar o tom**. É a diferença entre um atendente que responde tudo com o mesmo script e um que percebe "essa pessoa está com pressa e chateada — primeiro acalmo, depois resolvo".

6. O orquestrador (`chatbot_reply`)

A função que junta tudo — o "gerente" que coordena analista e redator:

```

def chatbot_reply(session, user_query):
    analysis = analyze_customer_query(user_query)          # 1º telefonema: entender
    reply = generate_personalized_response(session, user_query, analysis) # 2º telefonema:
    responder
    return {
        "customer_id": session.customer_id,
        "query": user_query,
        "analysis": analysis,
        "reply": reply,
    }

```

7. O arquivista (resumo + persistência)

Ao fim da conversa, uma **terceira chamada ao LLM** escreve um resumo, e o resultado vira um arquivo JSON padronizado em disco — um por conversa — mais um arquivo consolidado com todas:

```

{
  "timestamp": "2026-05-14T18:53:14",
  "customer_id": "CUST-78C546C3",
  "conversation_summary": "Cliente relatou pedido não entregue...",
  "query_category": "general_information",
  "customer_sentiment": "negative",
  "urgency_level": "high",
  "resolution_status": "escalated",
  "follow_up_required": true
}

```

💡 **Analogia** — é o fechamento do protocolo de atendimento: o que aconteceu, como o cliente saiu, se alguém precisa ligar de volta. Com milhares desses arquivos, a loja consegue medir: quais problemas mais aparecem? Quantos casos urgentes por semana?

O modo simulação (Mock Mode)

Cada chamada ao LLM custa dinheiro. Para desenvolver e testar sem gastar, o notebook tem um **modo simulação**: uma chave (`MOCK_MODE = True`) que troca as três funções que chamam a OpenAI por versões locais baseadas em regras simples (procurar palavras-chave como "refund", "receipt", "install" na mensagem).

💡 **Analogia** — é o simulador de voo do projeto: os pilotos treinam os procedimentos sem gastar combustível. As respostas do simulador são mais burras que as do LLM real, mas o *encanamento* (fluxo, arquivos, roteamento) é exercitado de verdade.

Validação: os 10 casos de teste

O notebook termina rodando 10 mensagens típicas de clientes e conferindo a classificação — de "o iPhone 15 tem em estoque?" (informação de produto, urgência baixa) até "**EMERGÊNCIA**: meu pedido nunca chegou e preciso do notebook amanhã" (urgência alta → encaminhado ao time prioritário).

Limitações que motivam a semana 2

Incômodo da semana 1	Consequência
JSON lido "na unha", com gambiarra de limpeza	Se o LLM mudar o formato, o programa quebra silenciosamente
Roteamento espalhado pelo código	Difícil de manter quando as categorias mudam
Resumo gasta uma 3ª chamada de LLM	Custo desnecessário — o resumo podia ser montado com dados que já temos
Nenhuma visibilidade interna	Se algo sai errado, ninguém sabe em qual etapa foi

A semana 2 ataca exatamente essa lista.

Passo a passo: como o código foi construído

Esta seção percorre o notebook **na ordem em que ele foi escrito**, célula por célula. É a reconstituição do trabalho: o que se digita primeiro, o que vem depois, e por quê.

Passo 1 — Preparar o terreno (importações e pastas)

Todo programa Python começa declarando as caixas de peças (bibliotecas) que vai usar:

```
import os                # conversar com o sistema operacional
import json              # ler/escrever o formato JSON
import uuid              # gerar códigos únicos (o protocolo do cliente)
from datetime import datetime # carimbos de data e hora
from pathlib import Path  # manipular pastas e arquivos

from openai import OpenAI # a biblioteca oficial da OpenAI

# Carrega o arquivo .env (onde mora a chave secreta da API)
from dotenv import load_dotenv
load_dotenv()

# Cria o cliente – ele lê a OPENAI_API_KEY do ambiente sozinho
client = OpenAI()

# Garante que a pasta de conversas salvas existe
DATA_DIR = Path("conversation_data")
DATA_DIR.mkdir(exist_ok=True) # exist_ok=True: se já existir, não reclama
```

Detalhe de segurança: a chave da API **não aparece no código**. Ela vive num arquivo `.env` separado, que fica fora do controle de versão (o `.gitignore` o exclui). Quem clonar o projeto no GitHub não leva a chave junto.

Passo 2 — Escrever a ficha da empresa (`COMPANY_CONTEXT`)

Um dicionário Python com os dados da loja (mostrado na seção anterior). Sem mistério técnico — o trabalho aqui foi **de conteúdo**: decidir o que um atendente precisa saber (horários, políticas, produtos, serviços, contato) e estruturar isso de forma organizada.

Passo 3 — Escrever o roteiro do atendente (`SYSTEM_ROLE`)

O prompt de sistema, um texto de ~40 linhas. As decisões de escrita que importam:

1. **Persona primeiro** — "You are a professional customer service assistant for TechStore Plus".
2. **Regras de comportamento** — educação, empatia com clientes chateados, adaptação de tom.
3. **A regra anti-invenção** — "Do not invent unavailable company policies" + "Use the company context below as your source of truth". Estas duas linhas são a diferença entre um bot confiável e um bot que promete frete grátis que não existe.

4. **A ficha injetada** — `{json.dumps(COMPANY_CONTEXT, indent=2)}` cola a ficha inteira dentro do texto.
5. **O fluxo em 5 etapas numeradas** — cumprimentar → coletar dados → identificar o tipo → rotear → dar próximos passos. LLMs seguem bem listas numeradas.

Passo 4 — Construir a pasta de atendimento (`ConversationSession`)

A classe de memória (mostrada na seção anterior). A ordem das decisões ao escrevê-la:

1. Cada sessão precisa de **identidade** → gerar `CUST-` + 8 caracteres aleatórios (`uuid`).
2. A lista de mensagens **nasce com o SYSTEM_ROLE dentro** → o roteiro sempre viaja com a conversa.
3. Dois métodos de anotação (`add_user_message` , `add_assistant_message`) → cada fala é registrada com seu papel (`role`), que é como a API da OpenAI distingue quem disse o quê.
4. Um método de leitura **sem o sistema** (`get_public_history`) → para exibir a conversa ao usuário ou resumi-la, as instruções internas não devem aparecer.

Passo 5 — O primeiro telefonema: o analista (`analyze_customer_query`)

A anatomia de uma chamada à API da OpenAI, que se repete no projeto inteiro:

```
response = client.chat.completions.create(    # "faça uma ligação"
    model="gpt-4o-mini",                    # com qual consultor falar
    temperature=0,                          # dial de criatividade (0 = burocrata)
    messages=[                              # o que dizer na ligação:
        {"role": "system", "content": "You are an expert...classifier..."}, # as instruções
        {"role": "user", "content": prompt} # o pedido
    ]
)
content = response.choices[0].message.content # a resposta vem aqui dentro
```

E o tratamento defensivo da resposta — a primeira lição de "LLMs desobedecem":

```
try:
    return json.loads(content)                # caminho feliz: o JSON veio limpo
except json.JSONDecodeError:
    # caminho triste: veio embrulhado
    cleaned = content.strip().replace("```json", "").replace("```", "").strip()
    return json.loads(cleaned)                 # desembrulha e tenta de novo
```

Passo 6 — O segundo telefonema: o redator (`generate_personalized_response`)

Três decisões de construção nesta função:

1. **A análise entra no prompt** — o redator recebe a ficha do analista (`json.dumps(analysis)`) e instruções condicionais ("SE o sentimento é negativo, comece com empatia; SE a urgência é alta, reconheça prioridade").

2. **O histórico inteiro vai junto** — `messages = session.messages + [instrução]`: o redator vê a conversa completa, por isso não repete perguntas.
3. `temperature=0.4` — um dedo de naturalidade. Textos para humanos não podem soar robóticos; classificações para máquinas não podem variar. Cada chamada tem seu dial.

Passo 7 — O gerente (`chatbot_reply`)

Quatro linhas que definem a arquitetura: analisar → responder → devolver tudo num pacote. A elegância está no que ela **não** faz: nenhuma lógica própria, só coordenação. Funções pequenas com um papel claro cada — o princípio que o projeto carrega até o fim.

Passo 8 — O terceiro telefonema: o arquivista (`generate_conversation_summary`)

```
summary_prompt = f"""
Summarize the following customer service conversation in one concise paragraph.

Conversation:
{json.dumps(public_history, indent=2)}          # o histórico SEM o system role
"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    temperature=0.2,                            # quase burocrata: resumo fiel, sem floreio
    messages=[...]
)
concise_summary = response.choices[0].message.content

# O resumo do LLM é UM CAMPO da ficha final – o resto vem da análise já feita:
conversation_json = {
    "timestamp": datetime.now().isoformat(timespec="seconds"),
    "customer_id": session.customer_id,
    "conversation_summary": concise_summary,        # ← único campo do LLM
    "query_category": latest_analysis.get("query_category", "general_information"),
    "customer_sentiment": latest_analysis.get("customer_sentiment", "neutral"),
    "urgency_level": latest_analysis.get("urgency_level", "low"),
    ...
    "resolution_status": resolution_status,        # informado por quem encerra o caso
    "follow_up_required": follow_up_required,
}
```

Repare nos `.get(campo, padrão)`: se a análise vier sem algum campo, o código usa um valor padrão em vez de quebrar. Programação defensiva de novo.

Passo 9 — Salvar e consolidar

```
def consolidate_conversations(output_file="consolidated_conversations.json"):
    all_conversations = []
    # Varre a pasta atrás de TODOS os arquivos conversation_*.json
    for file_path in DATA_DIR.glob("conversation_*.json"):
        with open(file_path, "r", encoding="utf-8") as file:
            all_conversations.append(json.load(file))

    consolidated_data = {
        "generated_at": datetime.now().isoformat(timespec="seconds"),
        "total_conversations": len(all_conversations),  # contagem automática
        "conversations": all_conversations,           # todas dentro
    }
    # ensure_ascii=False: preserva acentos legíveis no arquivo
    with open(output_path, "w", encoding="utf-8") as file:
        json.dump(consolidated_data, file, indent=2, ensure_ascii=False)
```

O padrão `with open(...)` garante que o arquivo é fechado mesmo se algo der errado no meio — o equivalente a "feche a gaveta ao terminar, aconteça o que acontecer".

Passo 10 — O modo simulação (a célula MOCK)

A célula mais extensa do notebook (~270 linhas) reescreve os três "telefonemas" como funções locais baseadas em regras. Dois exemplos do miolo:

```
def mock_extract_products(query):
    """Extrai produtos conhecidos por palavra-chave – substituí a IA no modo simulação."""
    product_keywords = {
        "iphone 15": "iPhone 15",
        "gaming headphones": "Gaming Headphones",
        "macbook pro": "MacBook Pro",
        ...
    }
    lower_query = query.lower()  # tudo minúsculo para comparar
    products = []
    for keyword, product_name in product_keywords.items():
        if keyword in lower_query and product_name not in products:
            products.append(product_name)  # achou? anota (sem duplicar)
    return products

def mock_extract_information(query):
    """Extrai nº de pedido, valores e datas com expressões regulares."""
    order_match = re.search(r"#?[A-Z]{3}-\d{4}-\d{3}", query)  # padrão TEC-2024-001
    amount_match = re.search(r"\$\s?\d+(?:\.\d{2})?", query)  # padrão $800 ou $59.99
    ...
```

Sobre a linha `r"#?[A-Z]{3}-\d{4}-\d{3}"`: é uma **expressão regular** — uma linguagem de padrões de texto. Lê-se: "um `#` opcional, 3 letras maiúsculas, hífen, 4 dígitos, hífen, 3 dígitos". É assim que o modo simulação encontra `#TEC-2024-001` no meio da frase sem nenhuma IA.

Com `MOCK_MODE = True`, as funções falsas **substituem** as verdadeiras (mesmos nomes, mesmas entradas e saídas) — o resto do notebook nem percebe a troca. Esse é o conceito de **interface estável**: peças intercambiáveis porque o encaixe é idêntico.

Passo 11 — Rodar de verdade (demonstração + 10 testes)

A célula de demonstração exercita o fluxo completo com o caso mais dramático:

```
session = ConversationSession()
user_query = "This is an emergency! My order #TEC-2024-001 never arrived and I need that laptop for work tomorrow!"

result = chatbot_reply(session, user_query)          # analisa + responde
summary = generate_conversation_summary(              # resume + estrutura
    session=session,
    latest_analysis=result["analysis"],
    resolution_status="escalated",                    # urgência alta → escalado
    actions_taken=[
        "Acknowledged urgent delivery issue",
        "Requested confirmation of delivery address",
        "Routed case to priority support",
    ],
    follow_up_required=True,
)
saved_file = save_conversation_json(summary)          # grava em disco
```

E a bateria final roda as **10 mensagens de teste** (uma por categoria/urgência) num laço, imprimindo a classificação de cada uma para conferência manual — o embrião dos testes automatizados que as semanas seguintes formalizam.

Capítulo 2 · Semana 2 — A linha de montagem (LangChain LCEL)

Arquivo: TechStorePlus_LangChain_LCEL_Chatbot.ipynb

O que foi construído

A semana 2 pega o mesmo atendente da semana 1 e o **reconstrói com ferramentas profissionais**. O que ele faz não muda muito — entender, responder, resumir, arquivar. *Como* ele faz muda completamente: em vez de funções soltas costuradas à mão, o fluxo vira uma **linha de montagem formal** usando a biblioteca **LangChain** e sua notação **LCEL** (*LangChain Expression Language*).

💡 **Analogia** — a semana 1 foi uma oficina de fundo de quintal: funciona, mas cada mecânico aperta os parafusos do seu jeito. A semana 2 transforma a oficina numa **fábrica com esteira**: cada estação faz uma coisa só, o carro passa de estação em estação, e há câmeras de inspeção em cada ponto. Se algo sair torto, sabemos exatamente em qual estação foi.

As três estações da esteira:

```
mensagem do cliente
↓
[Estação 1: ANÁLISE]      classifica a mensagem numa ficha validada (Pydantic)
↓
[Estação 2: RESPOSTA]    encaminha para 1 de 5 "personas especialistas" e gera a resposta
↓
[Estação 3: RESUMO]      monta o resumo final SEM chamar o LLM (de graça!)
↓
ficha final da conversa (ConversationSummary)
```

As melhorias, uma a uma

1. Adeus, gambiarra de JSON — chega o fiscal de qualidade (Pydantic)

Lembra do "plano B" da semana 1, que limpava manualmente o JSON que o LLM devolvia errado? A semana 2 elimina isso com **Pydantic**: em vez de *pedir* um formato e torcer, definimos um **molde rígido** e a biblioteca *garante* que a resposta se encaixa nele.

```

class ExtractedEntities(BaseModel):
    """Informações específicas extraídas da mensagem."""
    product_name: Optional[str] = Field(None, description='The specific product mentioned')
    order_number: Optional[str] = Field(None, description='The order number (e.g. #TEC-2024-001)')
    date: Optional[str] = Field(None, description='Any date mentioned')

class QueryAnalysis(BaseModel):
    """A ficha de classificação – agora com campos de preenchimento OBRIGATÓRIO e restrito."""
    query_category: Literal['technical_support', 'billing', 'returns',
                             'product_inquiry', 'general_information']
    urgency_level: Literal['low', 'medium', 'high']
    customer_sentiment: Literal['positive', 'neutral', 'negative']
    entities: ExtractedEntities

```

Tradução: o tipo `Literal[...]` significa "**só** estes valores são aceitos". Se o LLM tentar devolver `urgency_level: "muito alta"`, o programa rejeita na hora com um erro claro — em vez de aceitar silenciosamente e quebrar três etapas depois.

💡 **Analogia** — na semana 1, o formulário era de papel e o atendente podia escrever qualquer coisa nos campos. Agora o formulário é **digital com menus de seleção**: no campo "urgência" só existem três opções clicáveis. Ficou impossível preencher errado.

E a mágica que conecta o molde ao LLM é uma linha só:

```
analysis_chain = analysis_prompt | analysis_llm.with_structured_output(QueryAnalysis)
```

O `with_structured_output(QueryAnalysis)` diz ao LLM: "sua resposta DEVE se encaixar neste molde" — e a OpenAI força isso do lado de lá. A gambiarra de limpeza morreu.

2. O símbolo `|` — a esteira em notação de código

Repare no `|` (barra vertical) da linha acima. Na notação **LCEL**, ele significa "**e depois passa para**": `prompt | llm` lê-se "monte o prompt *e depois passe para* o LLM". É a esteira da fábrica escrita em código. A linha de montagem completa fica:

```

chain_with_context = (
    RunnablePassthrough.assign(          # Estação 1: adiciona o campo "analysis" à ficha
        analysis=RunnableLambda(lambda x: {'query': x['query']}) | analysis_chain
    )
    | RunnablePassthrough.assign(        # Estação 2: adiciona o campo "response"
        response=RunnableLambda(route_response)
    )
    | RunnablePassthrough.assign(        # Estação 3: adiciona o campo "summary"
        summary=RunnableLambda(build_summary)
    )
)

```

Tradução: `RunnablePassthrough.assign(...)` é uma estação que **acrescenta uma informação nova ao pacote que passa pela esteira**, sem apagar as anteriores. O pacote entra só com a pergunta do cliente e sai com pergunta + análise + resposta + resumo.

3. Cinco especialistas em vez de um generalista (`CATEGORY_PROMPTS`)

Na semana 1, um único prompt tentava dar conta de tudo. Agora existem **cinco prompts distintos, um por categoria**, cada um com sua persona e suas regras:

```
CATEGORY_PROMPTS = {
    'technical_support': ChatPromptTemplate.from_messages([
        ('system',
         'You are a TechStore Plus technical support specialist.\n'      # persona: técnico
         'Guidelines:\n'
         '- Begin with empathy if sentiment is negative.\n'            # comece com empatia
         '- Provide 2-3 concrete troubleshooting steps.\n'            # dê 2-3 passos práticos
         '- If urgency is high, offer escalation to a senior tech. '),  # urgente? escale
        ...
    ]),
    'billing': ChatPromptTemplate.from_messages([
        ('system',
         'You are a TechStore Plus billing specialist.\n'              # persona: financeiro
         '- Be professional and formal in tone.\n'                    # tom formal
         '- Reference the order number if mentioned. '),              # cite o nº do pedido
        ...
    ]),
    # ... + returns, product_inquiry, general_information
}
```

E a função `route_response` é o **porteiro** que lê a categoria na ficha de análise e entrega a mensagem ao especialista certo. Um técnico responde diferente de um vendedor — e agora isso está explícito e organizado num lugar só.

💡 **Analogia** — deixou de ser um balcão único e virou uma **central com ramais**: "para suporte técnico, ramal 1; para financeiro, ramal 2...". A diferença é que quem digita o ramal é o próprio sistema, baseado na classificação automática.

4. O resumo de graça (`build_summary`)

Melhoria de custo elegante: na semana 1, o resumo final era uma **terceira chamada ao LLM** (mais latência, mais centavos). Na semana 2, percebeu-se que todas as informações do resumo **já estavam em mãos** — categoria, sentimento, urgência, produtos, entidades vieram da Estação 1; a resposta veio da Estação 2. Então o resumo é **montado por código comum**, sem IA:


```
def build_summary(inputs):
    analysis = inputs['analysis']          # ficha da Estação 1 (já validada)
    response = inputs['response']          # resposta da Estação 2
    return ConversationSummary(
        timestamp=datetime.now().isoformat(),
        query_category=analysis.query_category,      # copia da análise
        customer_sentiment=analysis.customer_sentiment,
        urgency_level=analysis.urgency_level,
        resolution_status=STATUS_MAP[analysis.urgency_level], # regra fixa: alta→escalado,
        ...                                           # média→pendente, baixa→resolvido
    )
```

Tradução: uma chamada de LLM a menos por conversa = resposta mais rápida e mais barata, com resultado **determinístico** (sempre igual para a mesma entrada). Lição de arquitetura: **não use IA onde uma regra simples resolve.**

A regra fixa de status é um bom exemplo:

Urgência detectada	Status do caso
alta	escalated (escalado para humano)
média	pending (pendente)
baixa	resolved (resolvido)

5. As câmeras de inspeção (LangSmith)

A semana 2 também liga a **observabilidade**: com uma chave opcional, cada execução da esteira é gravada na plataforma **LangSmith** — que mostra, para cada conversa, quanto tempo e quantos **tokens** (a unidade de cobrança dos LLMs) cada estação consumiu, e o que exatamente entrou e saiu de cada uma.

```
# Precisa ser configurado ANTES de criar qualquer LLM — a biblioteca lê isso na criação
os.environ.setdefault('LANGCHAIN_PROJECT', 'Advanced-Customer-Agent')
if os.getenv('LANGCHAIN_API_KEY', '').strip():
    os.environ['LANGCHAIN_TRACING_V2'] = 'true'    # liga as câmeras
```

💡 **Analogia** — é a caixa-preta do avião + as câmeras da fábrica. Quando um cliente receber uma resposta estranha, dá para "rebobinar a fita" e ver: a classificação errou? O prompt do especialista estava ruim? O LLM alucinou? Sem isso, depurar um chatbot é adivinhação.

Semana 1 vs. Semana 2 — o placar

Aspecto	Semana 1 (na unha)	Semana 2 (LCEL)
Leitura do JSON	Manual, com gambiarra de limpeza	Pydantic valida automaticamente
Formato errado do LLM	Quebra silenciosa	Erro claro e imediato (<code>ValidationError</code>)
Roteamento	Espalhado pelo código	Encapsulado num roteador explícito
Resumo	3ª chamada de LLM (paga)	Montado por código (grátis)
Visibilidade	Nenhuma	LangSmith: tempo e custo por etapa
Categorias	8 genéricas	5 com persona especialista cada

O que ainda falta

O atendente já é organizado e inspecionável, mas ainda tem duas limitações grandes:

1. **Memória curta e sem consulta a sistemas** — ele lembra da conversa atual, mas não consegue *fazer* nada: não consulta pedidos, não agenda visita, não abre chamado.
2. **Só sabe o que está no prompt** — as políticas cabem numa ficha, mas e quando a base de conhecimento tiver 200 páginas de manuais?

A primeira limitação é atacada na **semana 3** (ferramentas + memória híbrida). A segunda, no **módulo 2 inteiro** (RAG).

Passo a passo: como o código foi construído

O notebook da semana 2 foi construído em três "componentes" + a montagem final — exatamente nesta ordem, com um teste isolado após cada componente.

Passo 1 — Setup: as câmeras ligam antes de tudo

```
from langchain_openai import ChatOpenAI # o LLM embrulhado pelo LangChain
from langchain_core.prompts import ChatPromptTemplate # moldes de prompt com lacunas
from langchain_core.runnables import RunnablePassthrough, RunnableLambda # peças da esteira
from pydantic import BaseModel, Field # o fiscal de formato

# ⚠️ ORDEM IMPORTA: o rastreamento precisa ser configurado ANTES de criar
# qualquer LLM — a biblioteca lê essas variáveis NO MOMENTO da criação.
os.environ.setdefault('LANGCHAIN_PROJECT', 'Advanced-Customer-Agent')
if os.getenv('LANGCHAIN_API_KEY', '').strip():
    os.environ['LANGCHAIN_TRACING_V2'] = 'true' # com chave → câmeras ligadas
else:
    os.environ['LANGCHAIN_TRACING_V2'] = 'false' # sem chave → roda normal, sem gravação
```

O *bug* que essa ordem evita: se o LLM fosse criado antes dessas linhas, o rastreamento silenciosamente não funcionaria — sem erro, sem aviso. Bugs de ordem de inicialização são dos mais traiçoeiros, e o notebook documenta isso em comentário.

Passo 2 — Componente 1: o classificador com molde (células 6-7)

Primeiro, os moldes (do menor para o maior — `ExtractedEntities` é usado dentro de `QueryAnalysis`):

```
class ExtractedEntities(BaseModel):
    """As informações pontuais extraídas da mensagem."""
    product_name: Optional[str] = Field(None, description='The specific product mentioned')
    order_number: Optional[str] = Field(None, description='The order number (e.g. #TEC-2024-001)')
    date: Optional[str] = Field(None, description='Any date mentioned')
```

Repare nos `description=`: essas descrições **não são comentários** — elas são enviadas ao LLM como parte do esquema, ensinando o que cada campo significa. Documentação que funciona.

Depois, o LLM e o prompt de análise — com uma decisão sutil de design:

```
analysis_llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# Prompt de sistema MINIMALISTA de propósito: sem dicas de categoria no texto,
# para não enviesar a classificação. A mensagem humana leva SÓ a pergunta crua.
analysis_prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert customer service query classifier...\n'
               'Categories: technical_support | billing | returns | product_inquiry | '
               'general_information\n'
               'Urgency: low | medium | high ...'),
    ('human', '{query}'), # ← a lacuna que será preenchida em tempo de execução
])

# A linha que une tudo — o primeiro trecho de LCEL do projeto:
analysis_chain = analysis_prompt | analysis_llm.with_structured_output(QueryAnalysis)
```

E imediatamente o **teste isolado** — antes de construir a próxima peça, prova-se que esta funciona:

```
test_result = analysis_chain.invoke({'query': 'This is an emergency! My order #TEC-2024-001 never arrived!'})
print(test_result.query_category)      # general_information
print(test_result.urgency_level)       # high
print(test_result.entities.order_number) # '#TEC-2024-001' ← extraído e validado
```

Repare: `test_result` não é um texto ou dicionário solto — é um **objeto** `QueryAnalysis` **validado**. Se o LLM tivesse devolvido lixo, essa linha teria explodido com um erro claro apontando o campo problemático.

Passo 3 — Componente 2: os cinco especialistas (célula 9)

A célula mais longa do notebook (~150 linhas) define os cinco prompts especialistas. O padrão de cada um:

```
'technical_support': ChatPromptTemplate.from_messages([
    ('system',
     'You are a TechStore Plus technical support specialist.\n'
     'Company context: {company_context}\n\n'           # a ficha da empresa entra por lacuna
     'Guidelines:\n'
     '- Begin with empathy if sentiment is negative.\n'
     '- Provide 2-3 concrete, actionable troubleshooting steps.\n'
     '- If urgency is high, offer immediate escalation to a senior technician.\n'
     '- End with a clear next step the customer should take.'),
    ('human',
     'Customer query: {query}\n'
     'Sentiment: {sentiment} | Urgency: {urgency} | Entities: {entities_json}\n\n'
     #           ↑ a análise do Componente 1 entra AQUI, campo a campo
     'Provide a helpful, empathetic technical support response with clear next steps.')
]),
```

Cada categoria tem seu tom calibrado: o técnico dá passos de solução; o financeiro é formal e cita o número do pedido; devoluções explica prazos; vendas recomenda produtos dentro do orçamento; o generalista acolhe o resto.

E o roteador — a função que escolhe o especialista em tempo de execução:

```
def route_response(inputs):
    analysis = inputs['analysis']      # a ficha que o Componente 1 acabou de produzir
    query = inputs['query']

    # Escolhe o prompt pela categoria; se vier algo desconhecido, cai no generalista
    prompt = CATEGORY_PROMPTS.get(analysis.query_category,
                                   CATEGORY_PROMPTS['general_information'])

    # Monta uma mini-esteira na hora (prompt escolhido | LLM) e executa
    chain = prompt | response_llm
    return chain.invoke({
        'query': query,
        'sentiment': analysis.customer_sentiment,
        'urgency': analysis.urgency_level,
        'entities_json': analysis.entities.model_dump_json(),
        'company_context': COMPANY_CONTEXT_STR,
    })
```

Por que uma função e não um `|` fixo? O comentário no próprio código explica: a esteira LCEL é **estática** — não sabe se ramificar sozinha. A decisão de qual prompt usar só existe **depois** que a análise rodou. Embrulhar a função em `RunnableLambda` a torna uma estação legítima da esteira, mantendo toda a lógica de desvio num único lugar. E o `.get(..., fallback)` garante: categoria desconhecida nunca quebra — cai no generalista.

Passo 4 — Componente 3: o resumo determinístico (célula 11)

Primeiro o molde do resultado final (`ConversationSummary`, espelhando o JSON da semana 1 — compatibilidade proposital: os arquivos das duas semanas convivem na mesma pasta). Depois, duas funções auxiliares pequenas e uma decisão de design:

```
def _short_action_from_response(response):
    """Mantém o resumo final conectado à resposta que foi de fato gerada."""
    text = _response_text(response).replace('\n', ' ').strip()
    return 'Generated customer response: ' + text[:180] + ('...' if len(text) > 180 else '')
```

O `_` no início do nome é a convenção Python para "função interna, não faz parte da interface pública" — organização silenciosa que facilita a manutenção.

E o `build_summary` copia os campos categóricos **da análise validada** (nunca re-perguntando ao LLM), aplica a regra fixa urgência→status, e carimba a hora. Zero tokens gastos.

Passo 5 — A montagem final (célula 13)

```
chain_with_context = (
    RunnablePassthrough.assign(
        analysis=RunnableLambda(lambda x: {'query': x['query']}) | analysis_chain
    )
    # após esta estação: {query, customer_id, analysis}
    | RunnablePassthrough.assign(
        response=RunnableLambda(route_response)
    )
    # após esta: {..., response}
    | RunnablePassthrough.assign(
        summary=RunnableLambda(build_summary)
    )
    # após esta: {..., summary}
)

# A cadeia "oficial" devolve só o resumo (o formato pedido pela entrega);
# a versão with_context existe para as demos poderem exibir também a resposta.
full_chain = chain_with_context | RunnableLambda(lambda x: x['summary'])
```

O detalhe do `Lambda x: {'query': x['query']}`: a `analysis_chain` espera receber **só** `{"query": ...}`, mas o pacote da esteira carrega mais coisas (`customer_id` etc.). Essa mini-função extrai apenas o campo necessário antes de entregar — um adaptador de tomada entre duas peças de encaixes diferentes.

Passo 6 — Rodar, salvar, consolidar

As células finais processam as mensagens de teste pela `full_chain`, salvam cada `ConversationSummary` como JSON individual (mesmo formato de arquivo da semana 1) e geram o consolidado:

```
saved_files = []
for item in results:
    file_path = save_conversation_json(item['result']) # um arquivo por conversa
    saved_files.append(file_path)

consolidated = consolidate_conversations() # o arquivo agregado
```

Com o LangSmith ligado, cada uma dessas execuções vira um **trace navegável**: as três estações, seus tempos, seus tokens, suas entradas e saídas — a régua com que as próximas semanas serão medidas.

Capítulo 3 · Semana 3 — Memória de longo prazo e mãos para trabalhar

Arquivos principais: `src/chains/memory_agent.py`, `src/components/hybrid_memory.py`, `src/components/customer_tools.py`, `src/mcp/notion_followup_server.py`

O que foi construído

Até aqui o atendente conversava bem, mas tinha duas deficiências graves para o mundo real:

1. **Memória de peixinho** — ele lembrava da conversa atual, mas conversas longas iam ficando caras (reenviar o histórico inteiro a cada mensagem) e eventualmente estouravam o limite do LLM.
2. **Mãos amarradas** — ele *falava* sobre pedidos, mas não conseguia *consultar* um pedido de verdade, nem abrir um chamado, nem agendar nada.

A semana 3 resolve os dois problemas e ainda adiciona um terceiro superpoder: criar **tarefas de follow-up automaticamente no Notion** (um aplicativo de organização usado por empresas).

Também é a semana em que o chatbot deixa de morar num notebook (arquivo de experimentos) e passa a morar em **módulos organizados** na pasta `src/` — código de gente grande, testável e reutilizável.

Peça 1 — A memória híbrida (`HybridMemory`)

O problema

Reenviar a conversa inteira a cada mensagem tem dois custos: dinheiro (paga-se por **token** enviado) e um teto físico — o LLM tem um limite de quanto texto consegue receber (a **janela de contexto**). As soluções tradicionais têm defeitos espelhados:

- **Janela deslizante** (manter só as últimas N mensagens): barata, mas o começo da conversa é **esquecido silenciosamente** — o cliente disse o número do pedido na mensagem 1 e o bot pergunta de novo na mensagem 15.
- **Só resumo**: mantém o fio da meada, mas perde a precisão do que foi dito palavra por palavra.

A solução: os dois ao mesmo tempo

MEMÓRIA HÍBRIDA de um cliente

[resumo acumulado]	← conversas antigas, condensadas por um LLM barato
[últimas 6 falas]	← palavra por palavra, precisão total
[ficha do cliente]	← nome, e-mail, VIP?, problemas anteriores (NUNCA é descartada)

```
BUFFER_SIZE = 6          # quantas falas recentes ficam guardadas palavra por palavra
MAX_TOKENS = 4_000       # orçamento máximo de texto enviado ao LLM por vez
```

```
class CustomerContext(BaseModel):
    """Ficha do cliente — vive FORA da conversa, então sobrevive a qualquer poda."""
    email: str
    name: str | None = None
    category: str | None = None          # ex.: "vip", "regular"
    preferences: list[str] = ...        # preferências conhecidas
    previous_issues: list[str] = ...    # problemas anteriores
```

Como funciona na prática: quando a conversa passa de 6 falas, as mais antigas são **destacadas do buffer e resumidas** por uma chamada barata de LLM; o resumo vai se acumulando num parágrafo que é sempre reenviado. O cliente pode conversar por horas — o bot nunca perde o fio, e o custo por mensagem fica controlado.

 **Analogia** — é como um médico numa consulta longa: ele não relê a gravação inteira das consultas anteriores; ele lê o **resumo do prontuário** (o sumário acumulado) + as **anotações da consulta atual** (o buffer) + a **ficha do paciente** (nome, alergias — que nunca sai da capa do prontuário).

Outro detalhe de projeto importante: **uma memória por cliente, sem compartilhamento**:

```
self._memories: dict[str, HybridMemory] = {}    # chave: e-mail do cliente

def _memory_for(self, email):
    if email not in self._memories:
        self._memories[email] = HybridMemory(customer_email=email)
    return self._memories[email]
```


Tradução: se as memórias se misturassem, o bot confundiria clientes e **vazaria dados pessoais** de um para outro. Cada e-mail tem sua gaveta trancada — o mesmo padrão usado em sistemas corporativos multi-cliente.

Peça 2 — As seis ferramentas (`customer_tools.py`)

Aqui o atendente ganha mãos. Uma **ferramenta** (*tool*) é uma função do nosso sistema que o LLM pode **decidir chamar** durante a conversa. As seis da TechStore:

Ferramenta	O que faz
<code>get_customer_info</code>	Busca o cadastro do cliente pelo e-mail
<code>get_order_status</code>	Consulta a situação de um pedido (ex.: TEC-2024-001)
<code>get_shipping_tracking</code>	Retorna código de rastreio e previsão de entrega
<code>get_customer_orders</code>	Lista todos os pedidos de um cliente
<code>get_customer_tickets</code>	Lista os chamados de suporte abertos do cliente
<code>create_support_ticket</code>	Abre um chamado novo de suporte

```
@tool
def get_order_status(order_number: str) -> str:
    """Look up the current status of an order by its order number (e.g. TEC-2024-001)."""
    # ↑ Esta descrição não é decorativa: é o TEXTO QUE O LLM LÊ para decidir
    # quando usar a ferramenta. Descrição ruim = ferramenta nunca usada.
    ...consulta o banco de dados e devolve a situação...
```

 **Analogia** — imagine dar ao consultor do telefone acesso a um painel com seis botões etiquetados. Ele lê as etiquetas e decide sozinho: "o cliente perguntou do pedido TEC-2024-001... vou apertar o botão *consultar pedido*". O LLM não executa nada — ele **pede** que o nosso sistema execute, recebe o resultado e continua a conversa com a informação em mãos.

As consultas batem num **banco de dados simulado** (`src/database/mock_db.py`) com clientes, pedidos e chamados fictícios — o suficiente para exercitar o fluxo completo sem depender de sistemas reais.

Peça 3 — O cérebro agêntico (`MemoryAgent` + **ReAct**)

Quem coordena tudo é o `MemoryAgent`, construído sobre o padrão **ReAct** (*Reason* + *Act* — raciocinar e agir), usando a peça pronta `create_react_agent` da biblioteca LangGraph:

```

class MemoryAgent:
    def __init__(self, ...):
        self._llm = ChatOpenAI(model="gpt-4.1-mini", temperature=0)
        # O loop ReAct pronto: pensar → usar ferramenta se precisar → pensar → responder
        self._agent = create_react_agent(self._llm, tools=TOOLS)
        self._memories = {} # uma memória por cliente

    def chat(self, customer_email, user_text):
        memory = self._memory_for(customer_email) # pega a gaveta do cliente
        memory.append_user(HumanMessage(content=user_text)) # anota a pergunta

        result = self._agent.invoke({"messages": memory.build_messages()})
        # ↑ o agente roda o ciclo: pode chamar 0, 1 ou várias ferramentas antes de responder

        reply = result["messages"][-1] # a resposta final
        memory.append_assistant(reply) # anota na gaveta
        ...

```

O ciclo ReAct em câmera lenta, para a pergunta *"cadê meu pedido TEC-2024-001?"*:

1. LLM pensa: "preciso consultar o pedido" → pede a ferramenta `get_order_status`
2. Sistema age: executa a consulta no banco → "pedido enviado, chega dia 20"
3. LLM pensa: "agora tenho a informação" → escreve a resposta final ao cliente

A *diferença filosófica*: até a semana 2, o **programador** definia o fluxo (analisar → rotear → responder, sempre). Agora o **LLM decide o próprio fluxo** — quantas ferramentas usar e em que ordem. Isso é o que define um **agente**.

Peça 4 — Follow-up automático no Notion (+ MCP)

Última peça: quando a conversa termina com uma pendência (*"vou verificar e te retorno"*), um detector identifica isso e **cria automaticamente uma tarefa no Notion** da equipe:

```

followup_task = detect_followup_task(customer_email, user_text, reply_text)
if followup_task is not None:
    task_client.create_task(followup_task) # nasce um card no Notion da equipe
    return f"{reply_text}\n\nNotion follow-up created."

```

E o arquivo `notion_followup_server.py` expõe essa mesma capacidade via **MCP** (*Model Context Protocol*) — um padrão aberto que funciona como uma **tomada universal para ferramentas de IA**: qualquer assistente compatível (como o Claude) pode se conectar a esse servidor e criar follow-ups da TechStore, sem código sob medida para cada integração.

💡 **Analogia** — o MCP está para as ferramentas de IA como o USB está para os periféricos: antes, cada impressora tinha seu cabo proprietário; com o padrão, qualquer dispositivo conversa com qualquer computador. O servidor MCP da TechStore é "um pendrive de criar tarefas" que qualquer IA compatível pode plugar.

Resumo da semana 3

Capacidade nova	Peça responsável
Conversas longas sem estourar custo/limite	HybridMemory (buffer + resumo + ficha)
Isolamento entre clientes	Uma memória por e-mail
Consultar pedidos, rastreio, cadastro, chamados	6 ferramentas @tool + banco simulado
Abrir chamado de suporte	create_support_ticket
Decidir sozinho quando usar cada ferramenta	Agente ReAct (create_react_agent)
Criar tarefas de follow-up para a equipe	Integração Notion + servidor MCP

Com isso o módulo 1 tem um atendente completo — que entende, lembra, age e registra. O passo seguinte é o desafio M1: **tirar isso do laboratório e colocar no ar**, como um sistema de verdade com site, login e banco de dados.

Passo a passo: como o código foi construído

A semana 3 é a primeira escrita fora de notebook — módulos Python em `src/`, cada um com um papel. A ordem de construção seguiu a dependência entre as peças: primeiro a memória (não depende de nada), depois as ferramentas (dependem do banco simulado), depois o agente (usa as duas), por fim as integrações.

Passo 1 — A memória híbrida (`hybrid_memory.py`), por dentro

O estado interno de cada memória é minimalista — três coisas:

```
def __init__(self, customer_email):
    self.context = CustomerContext(email=customer_email) # a ficha permanente do cliente
    self._buffer: list[BaseMessage] = []                # as falas recentes, verbatim
    self.running_summary: str = ""                       # o resumo acumulado (começa vazio)
```

O gatilho automático de resumo. Cada anotação verifica se o buffer estourou:

```
def append_user(self, message):
    self._buffer.append(message)
    if len(self._buffer) > BUFFER_SIZE:      # passou de 6 falas?
        self._summarise_displaced()          # → condensa as mais antigas
```

A condensação (_summarise_displaced) — o coração da memória:

```
def _summarise_displaced(self):
    displaced = self._buffer[:2]          # destaca o PAR mais antigo (cliente + bot)
    self._buffer = self._buffer[2:]      # o buffer encolhe de volta

    prompt = (
        "Update the running customer-service conversation summary.\n\n"
        f"Existing summary:\n{self.running_summary or 'No previous summary.'}\n\n"
        f"New turns to incorporate:\n{new_turns}\n\n"
        "Return a concise factual summary in 2-3 sentences. Preserve customer "
        "issues, order numbers, products, promised next steps, and unresolved questions."
    )
    # ↑ repare: o resumo é ATUALIZADO, não recriado – o LLM recebe o resumo
    # anterior + as falas novas e devolve a versão incorporada

    result = self._summariser.invoke([HumanMessage(content=prompt)])
    self.running_summary = result.content.strip()
```

Três decisões finas aqui: (1) o par completo é destacado junto — nunca fica uma pergunta sem a resposta; (2) o prompt lista **o que não pode ser perdido** (números de pedido, promessas, pendências) — instrução explícita contra o vício natural de resumos genéricos; (3) o resumidor é criado sob demanda (`if self._summariser is None`) — quem nunca passa de 6 falas nunca paga por ele.

A montagem final (build messages) — o que o agente de fato recebe a cada turno:

[illegible]

Passo 2 — As ferramentas (`customer_tools.py`)

Cada ferramenta segue o mesmo gabarito de 4 partes — vale ver uma completa:

```
@tool                                     # (1) o selo que a torna visível ao LLM
def get_order_status(order_number: str) -> str:      # (2) entrada e saída tipadas
    """Look up the current status of an order by its order number (e.g. TEC-2024-001).

    Returns the current status, or an explanatory message if not found.
    """
    order = MOCK_ORDERS.get(order_number.upper().lstrip("#"))
    if order is None:                            # (3) a descrição-vitrine que o LLM lê
        return f"No order found with number {order_number}." # (4) o caminho triste NUNCA explode:
    return (f"Order {order_number}: {order['status']}. "
            f"Items: {' '.join(order['items'])}. Total: ${order['total']}")
```

O detalhe do `.upper().lstrip("#")`: clientes digitam `#tec-2024-001`, `TEC-2024-001`, `tec-2024-001` ... a normalização aceita todas as variações. Pequenas gentilezas de robustez que separam demo de produto.

Passo 3 — O agente (`memory_agent.py`)

O fluxo do método `chat` — a espinha dorsal da semana — em versão anotada completa:

```

def chat(self, customer_email, user_text):
    # 1. Pega a gaveta de memória DESTA cliente (cria se for a primeira vez)
    memory = self._memory_for(customer_email)
    memory.append_user(HumanMessage(content=user_text))

    # 2. Desvio RAG (plugado na semana 4): pergunta "de biblioteca" nem passa
    # pelo agente – vai direto ao assistente de documentos
    if self._rag_assistant is not None and should_use_rag(user_text):
        reply_text = self._rag_assistant.answer(user_text)
        memory.append_assistant(AIMessage(content=reply_text))
        return reply_text

    # 3. O ciclo ReAct: o agente pensa, usa 0-N ferramentas, responde
    result = self._agent.invoke({"messages": memory.build_messages()})
    reply = result["messages"][-1] # a última mensagem é a resposta final

    # 4. Anota SÓ a resposta final na memória (não as idas-e-vindas de ferramenta –
    # isso manteria a memória inchada sem ganho)
    memory.append_assistant(reply)

    # 5. Pós-processamento: a conversa gerou uma pendência de follow-up?
    followup_task = detect_followup_task(customer_email, user_text, reply_text)
    if followup_task is None:
        return reply_text

    # 6. Tenta criar a tarefa no Notion – com DOIS níveis de fallback educado:
    if task_client is None:
        try:
            task_client = NotionTaskClient.from_env()
        except NotionConfigError:
            # sem credenciais? avisa, não quebra
            return f"{reply_text}\n\nNotion follow-up could not be created because " \
                "Notion credentials are not configured."

        try:
            task_client.create_task(followup_task)
        except Exception as exc:
            # Notion fora do ar? avisa, não quebra
            return f"{reply_text}\n\nNotion follow-up could not be created: {exc}"

    return f"{reply_text}\n\nNotion follow-up created."

```

O *padrão dos passos 5-6*: a resposta ao cliente **nunca** é sacrificada por uma falha na integração. O Notion pode estar mal configurado, fora do ar, sem permissão — o cliente recebe sua resposta de qualquer jeito, com um adendo transparente sobre o follow-up. Integrações externas são sempre tratadas como "podem falhar".

Passo 4 — O detector de follow-up (`followup_detector.py`)

Um classificador 100% baseado em regras (zero LLM) — e bilíngue:

```

FOLLOWUP_PHRASES = (
    "crie um follow-up", "criar um follow-up", "follow-up", "follow up",
    "me lembre", "lembre-me", "acompanhe", "verifique amanhã",
    "check tomorrow", "remind me",
)
# ← português E inglês

HIGH_PRIORITY_WORDS = ("urgente", "emergency", "asap", "imediato")

CATEGORY_KEYWORDS = (
    ("billing", ("refund", "invoice", "reembolso", "pagamento", "cobrança", ...)),
    ("returns", ("return", "devolução", "troca", ...)),
    ("technical_support", ("technical", "não liga", "power", "router", ...)),
    ...
)

```

A detecção: se a mensagem contém alguma frase-gatilho, nasce um `FollowUpTask` com prioridade (alta se contém palavra urgente) e categoria (pela primeira lista de palavras-chave que casar). Determinístico, instantâneo, testável — de novo o princípio: **regra onde regra basta**.

Passo 5 — O servidor MCP (`notion_followup_server.py`)

A construção é surpreendentemente curta — o padrão MCP faz o trabalho pesado:

```

from mcp.server.fastmcp import FastMCP

app = FastMCP("techstore-notion-followups")    # nasce o servidor, com nome

@app.tool()                                   # expõe a função como ferramenta MCP
def create_followup(task: str, customer_email: str, priority: str = "medium", ...):
    """Create a follow-up task in the TechStore Notion database."""
    followup = build_followup_task(task, customer_email, priority, ...)
    client = NotionTaskClient.from_env()
    return client.create_task(followup)

```

Qualquer assistente compatível com MCP que se conectar a esse processo ganha o botão "criar follow-up da TechStore" — sem uma linha de código de integração específica.

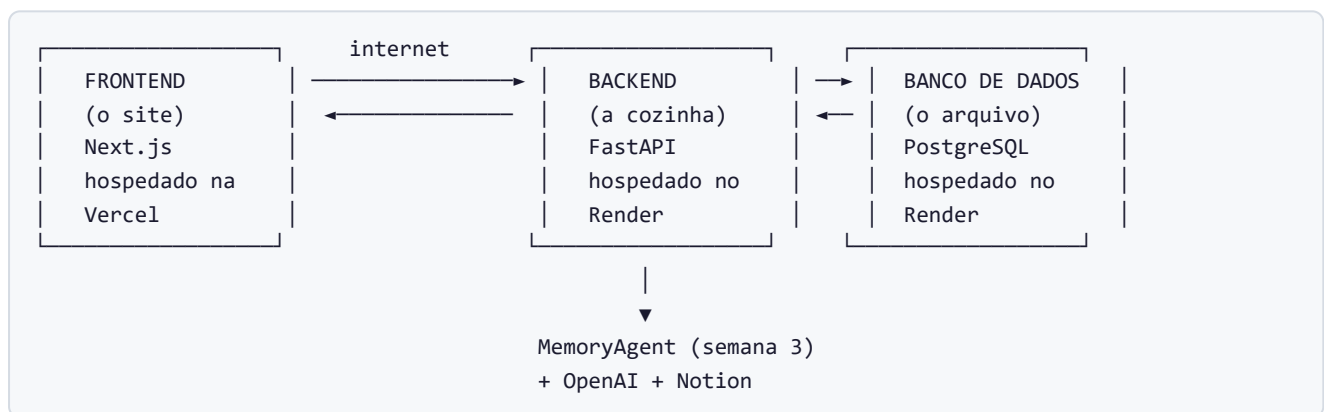
Capítulo 4 · Desafio M1 — Do laboratório para o ar

Repositório: `c03-t05-bruno-pieri-m1-challenge` · **Pastas principais:** `backend/`, `frontend/`, `render.yaml`

O que foi construído

Tudo até aqui vivia em *notebooks* — arquivos de laboratório onde o próprio desenvolvedor executa o código célula por célula. Ótimo para aprender; inútil para um cliente de verdade, que quer abrir um site, digitar e receber resposta.

O desafio M1 fecha essa lacuna: transforma o agente da semana 3 num **sistema em produção**, com três camadas independentes:



💡 **Analogia** — o restaurante ficou pronto. O **frontend** é o salão: mesas, cardápio, garçom — o que o cliente vê e toca. O **backend** é a cozinha: onde os pedidos são de fato preparados (é lá que mora o agente da semana 3). O **banco de dados** é a despensa e o livro de comandas: nada se perde quando o restaurante fecha e reabre. E cada parte funciona num prédio separado, alugado de um provedor de nuvem.

Peça 1 — O backend (FastAPI): o balcão de pedidos do nosso sistema

Na introdução vimos que **API** é o balcão de atendimento entre programas. Agora nós construímos o **nosso próprio balcão**, usando o framework **FastAPI**. Ele expõe "guichês" (chamados *endpoints*) na internet:

Guichê (endpoint)	O que faz
GET /health	Responde "estou vivo" — usado pelo provedor para monitorar o serviço
POST /api/auth/...	Valida a senha de acesso ao demo
POST /api/chat/sessions	Abre uma nova sessão de conversa para um e-mail
POST /api/chat/sessions/{id}/messages	Recebe uma mensagem e devolve a resposta do agente
GET /api/chat/sessions/{id}/messages	Lista o histórico da conversa

O coração do backend, comentado:

```

app = FastAPI(title="TechStore Plus Chat API")    # cria o "prédio" da API

app.add_middleware(
    CORSMiddleware,                               # porteiro de segurança do navegador:
    allow_origins=[settings.frontend_origin],     # só aceita pedidos vindos do NOSSO site
    ...
)

app.include_router(auth_router)                  # pendura o guichê de autenticação
app.include_router(chat_router)                 # pendura os guichês de chat

@app.get("/health")                              # o guichê "estou vivo"
def health():
    return {"status": "ok"}

```

Sobre o CORS: navegadores têm uma regra de segurança que impede o site A de fazer pedidos ao servidor B sem permissão explícita. O `CORSMiddleware` é onde declaramos "o site oficial da TechStore pode falar comigo; o resto, não".

Os contratos de entrada e saída

Cada guichê define **formulários rígidos** (Pydantic de novo!) para o que entra e o que sai:

```

class CreateSessionRequest(BaseModel):
    customer_email: EmailStr    # EmailStr valida que é um e-mail DE VERDADE
                                # "banana" é rejeitado antes de chegar ao agente

class SendMessageResponse(BaseModel):
    session_id: str            # o protocolo da conversa
    assistant_message: str     # a resposta do agente
    created_at: datetime       # carimbo de hora

```

Tradução: é a mesma filosofia da semana 2 (moldes rígidos), agora aplicada à fronteira com o mundo externo — onde ela é ainda mais importante, porque da internet chega de tudo.

A tranca da porta

O demo é protegido por senha. Cada pedido ao chat precisa trazer a senha num campo especial do cabeçalho:

```
def require_demo_password(x_demo_password: str = Header(default="")):
    if x_demo_password != get_settings().demo_password:
        raise HTTPException(status_code=401, detail="Invalid demo password.")
        # 401 = código universal de "não autorizado"

router = APIRouter(
    prefix="/api/chat",
    dependencies=[Depends(require_demo_password)],  # TODA rota de chat passa pela tranca
)
```

Tradução: o `Depends(...)` prende a verificação em **todos** os guichês de chat de uma vez — impossível esquecer a tranca numa porta nova.

A conexão com a semana 3

E onde entra o agente? Numa dobradiça mínima e elegante:

```
@lru_cache
def get_agent() -> Agent:
    return MemoryAgent()

# ← cria o agente UMA vez e reaproveita sempre
# (criar o agente é caro; reutilizar é grátis)
# ← o MESMO MemoryAgent da semana 3, sem alterações
```

Tradução: o backend não reinventa nada — ele **embrulha** o agente da semana 3 e o serve pela internet. É a recompensa por ter organizado o código em módulos: a peça encaixou sem retrabalho.

Peça 2 — O banco de dados (PostgreSQL): a memória que sobrevive

Até aqui, o histórico das conversas vivia na memória do programa — desligou, perdeu. Em produção isso é inaceitável. Entra o **PostgreSQL**, um banco de dados profissional, com duas "planilhas" (tabelas):

SESSIONS (as conversas)

session_id	customer_email
abc-123	ana@mail.com
def-456	ze@mail.com

MESSAGES (as falas)

id	session	role	content
1	abc-123	user	"Cadê..."
2	abc-123	assistant	"Seu..."



Cada fala de cada conversa vira uma linha permanente. O servidor pode reiniciar, cair, ser atualizado — as conversas continuam lá.

Peça 3 — O frontend (Next.js): o rosto do sistema

O `frontend/` contém o site em si, feito com **Next.js** (framework da linguagem TypeScript, prima do Python voltada para interfaces web). O fluxo do usuário:

1. Abre o site → tela pede a **senha do demo**.
2. Informa seu **e-mail** → o site pede ao backend para abrir uma sessão.
3. Digita mensagens numa **tela de chat** → cada mensagem viaja ao backend, que aciona o agente e devolve a resposta.

O frontend não tem inteligência nenhuma — de propósito. Ele é só a vitrine; toda a lógica mora no backend. Essa separação permite, por exemplo, trocar o site inteiro sem tocar no agente, ou criar um aplicativo de celular que usa o mesmo backend.

Peça 4 — O deploy (Render + Vercel): alugando os prédios

"**Deploy**" é o ato de publicar o sistema em servidores na internet. O projeto usa dois provedores com planos gratuitos:

- **Render** hospeda o backend + o banco PostgreSQL.
- **Vercel** hospeda o frontend.

E aqui entra um conceito bonito: **infraestrutura como código**. Em vez de clicar em dezenas de telas de configuração, o arquivo `render.yaml` descreve tudo que o Render precisa criar:

```
services:
  - type: web
    name: techstore-chat-api
    runtime: python
    plan: free # plano gratuito
    buildCommand: pip install -r backend/requirements.txt # como construir
    startCommand: uvicorn backend.app.main:app --port $PORT # como ligar
    healthCheckPath: /health # onde conferir se está vivo
    envVars:
      - key: DATABASE_URL
        fromDatabase: {name: techstore-chat-db, ...} # conecta ao banco automaticamente
      - key: OPENAI_API_KEY
        sync: false # segredo: preenchido à mão, NUNCA no código

databases:
  - name: techstore-chat-db # o banco também nasce daqui
    plan: free
```

Tradução: esse arquivo é a **planta baixa do restaurante**. Entregue a planta ao Render e ele constrói tudo: o serviço web, o banco, as conexões. Se um dia for preciso reconstruir do zero, a planta garante que sai idêntico. Repare no `sync: false` das chaves — os **segredos** (senhas, chaves de API) ficam fora do código e são informados direto no painel do provedor.

Peça 5 — Os testes: a rede de segurança

O backend tem uma bateria de testes automáticos (`pytest`): autenticação, envio de mensagens, persistência, configuração de CORS, o guichê de saúde. Eles rodam com um **agente falso** (que responde sempre a mesma coisa) — porque o objetivo é testar o *encanamento* do backend, não a inteligência.

💡 **Analogia** — antes de abrir o restaurante, o bombeiro testa os hidrantes com água comum — não precisa de um incêndio real. Os testes garantem que, quando o agente de verdade estiver plugado, os canos aguentam.

O teste de fumaça final

O README define o ritual de verificação pós-deploy (o *smoke test* — "ligou, saiu fumaça?"):

1. Abrir o site na Vercel → digitar a senha do demo
2. Informar um e-mail → mandar "*Hello, can you help me?*"
3. Conferir que o assistente responde
4. Pedir "*Pode criar um follow-up para verificar meu ticket amanhã?*"
5. Conferir que **um card aparece no Notion da equipe**

Se os cinco passos passam, o circuito completo está vivo: navegador → Vercel → Render → agente → OpenAI/Notion → banco → de volta ao navegador.

O que o módulo 1 entregou, em uma frase

Um atendente virtual que entende, lembra, consulta sistemas, age e registra — **acessível por qualquer pessoa com um navegador**, rodando em infraestrutura de nuvem descrita em código, com testes e monitoramento.

O módulo 2 ataca a fraqueza restante: o conhecimento do atendente ainda cabe num prompt. E quando a loja tiver manuais, políticas e catálogos de centenas de páginas?

Passo a passo: como o código foi construído

O backend segue uma **arquitetura em camadas** — cada pasta é uma camada com uma responsabilidade, e as camadas só conversam com as vizinhas. Construir de baixo para cima:

```
backend/app/
├─ db/          camada 1: o arquivo (modelos de tabela + repositório + conexão)
├─ core/        camada 2: configuração e segurança
├─ services/    camada 3: a ponte para o agente da semana 3
├─ api/         camada 4: os guichês HTTP (auth + chat)
└─ main.py     a montagem final
```

Passo 1 — As tabelas (`db/models.py`)

Com a biblioteca SQLAlchemy, cada tabela do banco é declarada como uma classe Python:

```
class ChatSession(Base):
    __tablename__ = "chat_sessions"                # nome da tabela no banco

    id: Mapped[str] = mapped_column(Text, primary_key=True,
                                     default=lambda: str(uuid4())) # id único automático
    customer_email: Mapped[str] = mapped_column(Text, nullable=False) # obrigatório
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utc_now)
    messages: Mapped[list["ChatMessage"]] = relationship(
        back_populates="session",
        cascade="all, delete-orphan",             # apagar a sessão apaga as mensagens dela
    )

class ChatMessage(Base):
    __tablename__ = "chat_messages"
    id: Mapped[str] = ...
    session_id: Mapped[str] = mapped_column(ForeignKey("chat_sessions.id"))
    #                                     ↑ a "linha de costura": toda mensagem
    #                                     aponta para a sessão a que pertence
    role: Mapped[str] = ...                 # "user" ou "assistant"
    content: Mapped[str] = ...              # o texto da fala
```

Dois detalhes de qualidade: o `utc_now()` usa fuso **UTC** (o padrão universal — servidores em países diferentes não podem discordar da hora); e o `cascade="all, delete-orphan"` garante que não sobram mensagens órfãs se uma sessão for apagada.

Passo 2 — O repositório (`db/repository.py`)

O padrão **Repository** concentra todo o acesso ao banco num único lugar — nenhuma outra camada escreve consultas:

```

class ChatRepository:
    def create_session(self, customer_email):
        session = ChatSession(customer_email=customer_email)
        self.db.add(session)          # prepara a inserção
        self.db.commit()              # grava de fato (a "assinatura no cartório")
        self.db.refresh(session)      # relê o registro (agora com id e timestamps)
        return session

    def add_message(self, session_id, role, content):
        message = ChatMessage(session_id=session_id, role=role, content=content)
        session = self.get_session(session_id)
        if session is not None:
            session.updated_at = utc_now()    # a sessão "acorda": atualiza o carimbo
        self.db.add(message)
        self.db.commit()
        ...

    def list_messages(self, session_id):
        return (self.db.query(ChatMessage)
                .filter(ChatMessage.session_id == session_id)  # só desta conversa
                .order_by(ChatMessage.created_at.asc())         # em ordem cronológica
                .all())

```

Por que isolar? Se um dia o banco mudar (PostgreSQL → outro), ou uma consulta precisar de otimização, **um arquivo** muda. E os testes conseguem substituir o repositório inteiro por um duplê.

Passo 3 — Configuração e segurança (`core/`)

`config.py` lê as variáveis de ambiente (URL do banco, senha do demo, origem permitida do CORS...) numa classe de settings validada. `security.py` (mostrado na seção anterior) implementa a tranca de senha. Separar configuração do código é o que permite o **mesmo código** rodar no laptop (com SQLite e senha "demo") e no Render (com PostgreSQL e segredos reais) sem nenhuma alteração.

Passo 4 — A ponte para o agente (`services/chat_service.py`)

O arquivo inteiro tem 16 linhas — e é uma aula de encaixe limpo:

```

from src.chains.memory_agent import MemoryAgent    # ← importa a semana 3

class Agent(Protocol):
    def chat(self, customer_email, user_text): ... # o "contrato": qualquer coisa com
                                                    # um método chat() serve

@lru_cache
def get_agent() -> Agent:                          # memoização: cria UMA vez, reusa sempre
    return MemoryAgent()

```

O `Protocol` é o pulo do gato dos testes: os endpoints não exigem um `MemoryAgent` — exigem "algo que tenha `.chat()`". Nos testes, entra um duplê que responde instantaneamente e de graça. Em produção, entra o agente real. Mesmo encaixe.

Passo 5 — O guichê principal (`api/chat.py`)

O endpoint de envio de mensagem, o coração do sistema, anotado linha a linha:

```
@router.post("/sessions/{session_id}/messages", response_model=SendMessageResponse)
def send_message(
    session_id: str,                    # vem da URL
    payload: SendMessageRequest,        # vem do corpo do pedido (validado!)
    db: Session = Depends(get_db),      # o FastAPI INJETA a conexão de banco
    agent: Agent = Depends(get_agent),  # ...e o agente (real ou dublê)
):
    repo = ChatRepository(db)
    session = repo.get_session(session_id)
    if session is None:
        raise HTTPException(status_code=404, detail="Chat session not found.")
        # ↑ sessão inexistente? 404 claro, nunca um erro genérico

    user_message = payload.message.strip()
    if not user_message:
        raise HTTPException(status_code=422, detail="Message cannot be empty.")
        # ↑ mensagem vazia? 422 explicando o problema

    repo.add_message(session_id, "user", user_message)          # 1. grava a pergunta
    assistant_message = agent.chat(session.customer_email,      # 2. aciona o agente
                                   user_message)                #   (semana 3 inteira roda aqui)
    saved_reply = repo.add_message(session_id, "assistant",      # 3. grava a resposta
                                   assistant_message)
    return SendMessageResponse(                                  # 4. devolve ao site
        session_id=session_id,
        assistant_message=assistant_message,
        created_at=saved_reply.created_at,
    )
```

A *ordem gravar-perguntar-gravar importa*: a pergunta do cliente é persistida **antes** de acionar o agente. Se o agente falhar no meio, a pergunta não se perde — o histórico conta a verdade.

Sobre o `Depends(...)`: é a **injeção de dependências** do FastAPI — o framework entrega a cada pedido uma conexão de banco fresca e o agente compartilhado. O endpoint não sabe *de onde* eles vêm; só declara *do que precisa*. É isso que torna cada guichê testável isoladamente.

Passo 6 — A montagem (`main.py`) e o ciclo de vida

```
app = FastAPI(title="TechStore Plus Chat API")
app.add_middleware(CORSMiddleware, allow_origins=[settings.frontend_origin], ...)
app.include_router(auth_router)      # pendura os guichês de autenticação
app.include_router(chat_router)      # pendura os guichês de chat

@app.on_event("startup")
def startup():
    create_tables()                  # ao ligar: cria as tabelas se não existirem

@app.get("/health")
def health():
    return {"status": "ok"}          # o "estou vivo" que o Render consulta
```

Passo 7 — Os testes do encanamento

A bateria de testes cobre cada guichê com o agente-dublê. Exemplo do padrão (de `test_chat_api.py`):

```
def test_send_message_returns_assistant_reply(client):
    # 1. abre uma sessão
    created = client.post("/api/chat/sessions",
                          json={"customer_email": "ana@example.com"},
                          headers={"X-Demo-Password": "demo"})
    session_id = created.json()["session_id"]

    # 2. envia uma mensagem
    response = client.post(f"/api/chat/sessions/{session_id}/messages",
                          json={"message": "Hello!"},
                          headers={"X-Demo-Password": "demo"})

    # 3. confere o contrato: status 200, resposta presente, persistência ok
    assert response.status_code == 200
    assert response.json()["assistant_message"]
```

Sem senha → 401. Sessão inexistente → 404. Mensagem vazia → 422. Cada regra do guichê tem seu teste — e o conjunto roda em segundos, sem tocar na OpenAI.

PARTE II — Módulo 2: O Atendente que Estuda (RAG)

Capítulo 5 · Semana 4 — Ensinando o atendente a consultar documentos

Arquivos: `Week4_RAG_TechStore.ipynb`, `Week4_RAG_Python_Library.ipynb`, pasta `src/rag/`

O problema que abre o módulo 2

O atendente do módulo 1 sabe **o que está no prompt** — aquela ficha da empresa com meia dúzia de políticas. Mas uma empresa real tem manuais de produto, tabelas de garantia, guias de solução de problemas, políticas com letras miúdas... **centenas de páginas**. Não cabe tudo no prompt (limite físico), e mesmo se coubesse, custaria uma fortuna reenviar tudo a cada mensagem.

A resposta da indústria para isso tem nome: **RAG** — *Retrieval-Augmented Generation*, ou "geração aumentada por busca". A ideia em uma frase:

Em vez de fazer o atendente **decorar** a biblioteca inteira, ensinamos ele a **consultar** a biblioteca — achar as 3 ou 4 páginas relevantes para a pergunta e responder **só com base nelas**.

💡 **Analogia** — é a diferença entre uma prova decoreba e uma **prova com consulta**. O atendente RAG não precisa saber a política de devolução de cor; ele precisa saber *onde procurar* e *como ler*. De bônus, quando a política mudar, basta trocar o documento na estante — não é preciso "retreinar" ninguém.

O fluxo RAG em cinco passos

FASE DE PREPARO (feita uma vez, "montando a biblioteca"):

1. CARREGAR docs da loja (.md, .txt, .pdf) → `document_loader.py`
2. PICAR em pedaços de ~700 caracteres (chunks) → `text_splitter.py`
3. INDEXAR cada pedaço vira um "endereço numérico" (embedding) e vai para a estante ChromaDB → `vector_store.py`

FASE DE USO (a cada pergunta do cliente):

4. BUSCAR os 4 pedaços mais parecidos com a pergunta → `similarity_search`
5. RESPONDER o LLM responde SÓ com base nesses pedaços, citando as fontes → `rag_chain.py`

Passo a passo, com o código real

Passo 1-2 — Picar os documentos (`text_splitter.py`)

Por que picar? Porque a busca funciona melhor com trechos focados: se a pergunta é sobre prazo de reembolso, queremos achar **o parágrafo** do reembolso, não o manual inteiro de 40 páginas diluindo a relevância.

```
DEFAULT_CHUNK_SIZE = 700      # cada pedaço tem ~700 caracteres (~1 parágrafo bom)
DEFAULT_CHUNK_OVERLAP = 120   # pedaços vizinhos compartilham 120 caracteres

splitter = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    separators=["\n\n", "\n", ". ", " ", ""],  # preferência de onde cortar:
)                                              # parágrafo > linha > frase > palavra
```

Dois detalhes espertos:

- **A sobreposição (overlap)** de 120 caracteres: se uma informação importante caísse exatamente na fronteira de dois pedaços, ela seria cortada ao meio e perdida. A sobreposição garante que fronteiras sempre existam **inteiras em pelo menos um pedaço**.
- **A ordem dos separadores**: o cortador tenta primeiro cortar em quebras de parágrafo (limpo), só recorrendo a cortes no meio de frases em último caso — como fatiar um bolo respeitando as camadas.

Passo 3 — A estante mágica (`vector_store.py` + ChromaDB)

Aqui entra o conceito mais bonito do módulo: o **embedding**. Um modelo especial (aqui, o `text-embedding-3-small` da OpenAI) converte qualquer texto numa **lista de ~1500 números** que funciona como o "endereço do significado" daquele texto. Textos que falam da mesma coisa ganham endereços próximos — **mesmo sem usar as mesmas palavras**.

```

class TechStoreVectorStore:
    def __init__(self, ...):
        # o tradutor de texto → números
        self.embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
        # a estante persistente no disco (pasta chroma_db/)
        self._client = chromadb.PersistentClient(path=str(self.persist_directory))
        self._collection = self._client.get_or_create_collection(name=self.collection_name)

    def add_documents(self, documents):
        texts = [d.page_content for d in documents]
        self._collection.upsert(
            ids=ids,
            documents=texts,
            metadatas=[d.metadatas for d in documents],
            embeddings=self.embeddings.embed_documents(texts),
        )

    def similarity_search(self, query, *, k=4):
        # converte a PERGUNTA para o mesmo sistema de endereços...
        # ...e pede à estante os k pedaços com endereços mais próximos
        result = self._collection.query(
            query_embeddings=[self.embeddings.embed_query(query)],
            n_results=k,
        )

```

💡 **Analogia** — uma biblioteca comum organiza livros por ordem alfabética: "reembolso" e "devolução do dinheiro" ficam em estantes distantes, apesar de serem o mesmo assunto. A estante vetorial organiza por **assunto no espaço**: tudo que fala de devolver dinheiro fica na mesma prateleira, não importa a palavra usada. Quando chega a pergunta "*quero meu dinheiro de volta*", o bibliotecário calcula o "endereço" da pergunta e vai direto à prateleira certa.

Isso resolve o problema clássico de buscas por palavra exata: o cliente pergunta "*posso trocar?*" e o documento diz "*política de devolução*" — a busca vetorial encontra mesmo assim, porque o **significado** é vizinho.

Passo 4-5 — Responder com consulta (`rag_chain.py`)

A peça final junta busca + LLM, com as duas regras de ouro do RAG no prompt:

```
def answer(self, question):
    documents = self._retriever.similarity_search(question, k=self._k) # busca os 4 melhores
    if not documents:
        return self._not_found_message # nada achado? admite, não inventa

    context = _format_context(documents) # cola os 4 trechos num bloco numerado
    prompt = (
        f"{self._system_prompt} "
        f"Answer the question using only the context below. " # REGRA 1: responda SÓ
        f"If the context does not contain the answer, say: " # com base no contexto
        f"{self._not_found_message}\n\n" # REGRA 2: se não estiver
        f"Context:\n{context}\n\n" # lá, diga "não sei"
        f"Question: {question}"
    )
    response = self._llm.invoke([HumanMessage(content=prompt)])
    sources = _format_sources(documents)
    return f"{response.content}\n\nSources: {sources}" # cita as fontes!
```

Tradução: o LLM recebe a pergunta **grampeada nos 4 trechos encontrados** e é instruído a não usar nada além deles. E a resposta sai com a lista de fontes — o cliente (ou o auditor) pode conferir de onde veio cada afirmação. É o principal antídoto contra a **alucinação** (o LLM inventar resposta com confiança).

A integração com o chatbot — o porteiro do RAG

Nem toda pergunta precisa da biblioteca ("oi, tudo bem?" não requer consulta). Uma função simples decide:

```
RAG_KEYWORDS = {"return", "refund", "warranty", "shipping", "policy",
                 "product", "troubleshoot", "manual", "support", ...}

def should_use_rag(user_text):
    """Se a mensagem menciona algum assunto 'de biblioteca', consulta o RAG."""
    return any(keyword in user_text.lower() for keyword in RAG_KEYWORDS)
```

E o `MemoryAgent` da semana 3 ganhou este desvio: perguntas "de biblioteca" vão para o assistente RAG; o resto segue o fluxo normal do agente. As duas capacidades convivem.

O mini-projeto: RAG sobre documentação da biblioteca Requests

Para provar que a receita é **genérica**, a semana 4 inclui um segundo notebook que monta o mesmo pipeline sobre outro acervo: a documentação oficial da *Requests* (uma biblioteca Python famosa). Mesmos passos — baixar páginas, picar, indexar (numa coleção ChromaDB separada), responder perguntas técnicas — e um bônus importante: **métricas de qualidade de busca** (precisão, revocação, F1 e curva ROC), medindo *quantos dos trechos recuperados eram de fato relevantes*.

Esse bônus planta a semente da semana 5: **como saber se o RAG está bom?** Não basta "parece que funciona" — é preciso medir.

Resumo do que a semana 4 entregou

Peça	Papel na "biblioteca"
<code>document_loader.py</code>	Recebe os livros (<code>.md</code> , <code>.txt</code> , <code>.pdf</code>)
<code>text_splitter.py</code>	Pica em trechos de ~700 caracteres com sobreposição
<code>vector_store.py</code> + ChromaDB	A estante que organiza por significado
<code>rag_chain.py</code>	O bibliotecário: busca, responde só com base no texto, cita fontes
<code>should_use_rag</code>	O porteiro: decide quais perguntas vão à biblioteca
Mini-projeto Requests	Prova que a receita serve para qualquer acervo + primeiras métricas

A biblioteca está de pé — mas é uma biblioteca ingênua: a busca às vezes traz 4 trechos quase idênticos, o trecho certo às vezes fica em 5º lugar (fora do top 4), e ninguém mediu sistematicamente a qualidade. A **semana 5** existe para afinar exatamente isso.

Passo a passo: como o código foi construído

A semana 4 foi construída como uma **caixa de peças em `src/rag/`** — cinco módulos pequenos, cada um dono de uma etapa — e um notebook que os encadeia e demonstra. Ordem de construção: da entrada (carregar) para a saída (responder).

Passo 1 — O recebedor de livros (`document_loader.py`)

```
SUPPORTED_TEXT_EXTENSIONS = {".md", ".txt"}

def load_document(path):
    """Carrega UM documento, com os metadados que a busca vai precisar."""
    document_path = Path(path)
    if not document_path.exists():
        raise FileNotFoundError(f"Document not found: {document_path}")    # falha clara e cedo

    extension = document_path.suffix.lower()
    if extension in SUPPORTED_TEXT_EXTENSIONS:
        return [_load_text_document(document_path)]    # texto: leitura direta
    if extension == ".pdf":
        return [_load_pdf_document(document_path)]    # PDF: uma página = um Document
    raise ValueError(f"Unsupported document type: {extension}")
```

O detalhe central é que **todo documento sai carimbado**:

```
def _load_text_document(path):
    return Document(
        page_content=path.read_text(encoding="utf-8").strip(), # o texto
        metadata={
            "source": str(path), # de onde veio (para as citações!)
            "title": path.name, # o nome amigável
            "document_type": path.suffix.lower().lstrip("."), # md/txt/pdf
        },
    )
```

Esses metadados são a matéria-prima das citações lá na frente: quando a resposta disser "Sources: policy_returns.md", é este carimbo, colocado na hora do carregamento, que viajou pelo pipeline inteiro. E no PDF, cada página ainda ganha `page_number` — citação com página.

Note também o tratamento do PDF: a biblioteca de PDF só é importada **dentro** da função (`import` tardio), com uma mensagem de erro que ensina a instalar o que falta. Quem só usa `.md` nunca paga o custo da dependência de PDF.

Passo 2 — O picador (`text_splitter.py`)

Já dissecado na seção anterior — 26 linhas, uma função. O acréscimo do passo a passo é o **carimbo de posição**:

```
chunks = splitter.split_documents(documents)
for index, chunk in enumerate(chunks):
    chunk.metadata = {**chunk.metadata, "chunk_index": index}
    #           ↑ preserva os metadados originais E adiciona a posição
```

O `chunk_index` vai ser usado no passo 3 para dar a cada pedaço uma **identidade estável**.

Passo 3 — A estante (`vector_store.py`)

A classe `TechStoreVectorStore` embrulha o ChromaDB. As decisões de construção:

Identidade estável dos pedaços — a função que gera a etiqueta de cada chunk:

```
def _document_id(document, index):
    source = str(document.metadata.get("source", "unknown"))
    chunk_index = document.metadata.get("chunk_index", index)
    return f"{source}:{chunk_index}" # ex.: "docs/kb/policy_returns.md:3"
```

Por que isso importa: o método de gravação usa `upsert` (update + insert) — se o mesmo documento for indexado duas vezes, os pedaços **substituem** os antigos em vez de duplicar. Rodar o preparo de novo é seguro; a estante nunca acumula cópias fantasmas. (Termo técnico: a operação é *idempotente*.)

A tradução de volta — a busca devolve dados crus do ChromaDB, e o código os re-embrulha em `Document` para o resto do pipeline não precisar conhecer o formato interno do banco:

```
def similarity_search(self, query, *, k=4):
    result = self._collection.query(
        query_embeddings=[self.embeddings.embed_query(query)], # pergunta → números
        n_results=k,
    )
    documents = result.get("documents", [[]])[0] # defensivo: se vier vazio, lista vazia
    metadatas = result.get("metadatas", [[]])[0]
    return [Document(page_content=content, metadata=metadata or {})
            for content, metadata in zip(documents, metadatas)]
```

Passo 4 — O bibliotecário (`rag_chain.py`)

Dissecado na seção anterior. O acréscimo aqui é a estrutura interna do contexto — como os 4 trechos são "grampeados" para o LLM:

```
def _format_context(documents):
    parts = []
    for index, document in enumerate(documents, start=1):
        title = document.metadata.get("title") or document.metadata.get("source", "Unknown")
        parts.append(f"[{index}] {title}\n{document.page_content}")
        #         ↑ cada trecho numerado e identificado – o LLM consegue
        #         referenciar "[2]" e o leitor consegue conferir
    return "\n\n".join(parts)

def _format_sources(documents):
    sources = []
    for document in documents:
        source = document.metadata.get("title") or document.metadata.get("source", ...)
        if source not in sources: # deduplicação: cada fonte citada UMA vez
            sources.append(str(source))
    return ", ".join(sources)
```

Passo 5 — O indexador de linha de comando (`index_knowledge_base.py`)

Um script de 31 linhas que junta os passos 1-3 num comando único: carregar `docs/knowledge_base/` → picar → gravar na estante. É o botão "reconstruir a biblioteca" — roda uma vez no preparo, ou sempre que os documentos mudarem.

Passo 6 — O notebook demonstra o ciclo completo

As células do `Week4_RAG_TechStore.ipynb` seguem os títulos: **1. Load Documents** → **2. Split Into Chunks** → **3. Generate Embeddings And Store In ChromaDB** → **4. Reload And Retrieve** → **5. Ask Grounded Questions**.

A célula 4 tem um teste sutil e importante: ela **recarrega a estante do disco** (nova instância apontando para a mesma pasta `chroma_db/`) antes de buscar — provando que a persistência funciona

de verdade, e não só enquanto o objeto original está na memória.

Passo 7 — O mini-projeto Requests (generalização + métricas)

O segundo notebook (`Week4_RAG_Python_Library.ipynb`) reusa as mesmas peças sobre outro acervo, com dois módulos novos:

- `python_library_docs.py` — baixa páginas selecionadas da documentação oficial da Requests e guarda cópias `.txt` locais (o acervo não depende de internet depois do preparo).
- `evaluation.py` — as primeiras métricas do projeto: para um conjunto de perguntas com gabarito (quais documentos são relevantes para cada uma), calcula **precisão** (dos recuperados, quantos eram certos?), **revocação** (dos certos, quantos foram recuperados?), **F1** (a média harmônica dos dois) e os pontos da **curva ROC**.

E um bônus de usabilidade: `scripts/requests_rag_chatbot.py` empacota tudo num chatbot de terminal (`python scripts/requests_rag_chatbot.py --question "How do I set a timeout?"`) — a primeira interface "de usuário" do RAG.

Capítulo 6 · Semana 5 — Afinando a biblioteca (otimização do RAG)

Arquivo: `Week5_RAG_Optimization.ipynb` (usando o código do repositório `c03-t05-bruno-pieri-m2-challenge`)

O que foi construído

A biblioteca da semana 4 funciona, mas tem três vícios de biblioteca nova:

1. **Traz 4 cópias do mesmo assunto** — se a pergunta é sobre garantia, a busca por similaridade tende a devolver 4 trechos quase iguais da mesma seção, desperdiçando espaço de contexto.
2. **A ordem nem sempre é a melhor** — o trecho *realmente* certo às vezes vem em 3º ou 4º lugar (ou fica de fora).
3. **Ninguém mediu nada** — "parece melhor" não é critério de engenharia.

A semana 5 ataca os três com quatro componentes: **MMR**, **re-ranking com cross-encoder**, um **experimento de tamanho de pedaço**, e **métricas objetivas** comparando antes × depois.

Componente 1 — MMR: o bibliotecário que evita repetição

MMR (*Maximal Marginal Relevance*) muda o critério da busca: em vez de "os 4 trechos mais parecidos com a pergunta", passa a ser "os trechos mais parecidos com a pergunta **que sejam diferentes entre si**".

 **Analogia** — você pede ao bibliotecário material sobre "garantia". O bibliotecário ingênuo traz 4 fotocópias de páginas vizinhas do mesmo capítulo. O bibliotecário MMR traz: 1 página do capítulo de garantia, 1 da política de garantia estendida, 1 do procedimento de acionamento e 1 da tabela de prazos — **cobertura**, não redundância.

O notebook prova isso com uma consulta propositalmente ampla:

```
broad_query = "TechStore Plus products support warranty return policy"
mmr_docs = retriever.invoke(broad_query)

# Verificação: os resultados vêm de arquivos DISTINTOS?
unique_sources = len({d.metadata['source'] for d in mmr_docs})
print("✓ Diversity check passed" if unique_sources >= 3 else "⚠ Low diversity")
```

Como o MMR funciona por dentro: ele primeiro pesca um lote maior de candidatos (ex.: 20), depois monta o resultado um a um — cada escolha pontua **relevância para a pergunta** menos **semelhança com o que já foi escolhido**. Ganha quem agrega informação nova.

Componente 2 — Re-ranking com cross-encoder: a segunda opinião

A busca vetorial é rápida mas aproximada — ela compara "endereços de significado" calculados **separadamente** para pergunta e documento. O **cross-encoder** é um modelo mais lento e mais preciso que lê **pergunta e trecho juntos, lado a lado**, e dá uma nota de relevância real para o par.

A arquitetura em dois estágios usa cada um no que é bom:

```
pergunta do cliente
↓
[Estágio 1 – MMR]          rápido, pesca 6 candidatos diversos na estante inteira
↓
[Estágio 2 – Cross-encoder] lento porém preciso, lê os 6 com atenção e reordena
↓
top-3 vão para o LLM      (contexto menor E melhor)
```

```
mmr_candidates = retriever.invoke(rerank_query)    # estágio 1: 6 candidatos
top_docs = rerank(rerank_query, mmr_candidates)    # estágio 2: reordena, fica com top-3

# Cada trecho sai com sua nota de relevância:
# [1] score=+5.1042 | policy_extended_warranty.txt
# [2] score=+3.8871 | policy_warranty.txt
# [3] score=-1.2094 | product_catalog.txt
```

 **Analogia** — é um processo seletivo em duas fases: a triagem de currículos (rápida, olha palavras-chave, elimina 95%) e a entrevista presencial (lenta, mas avalia de verdade — só para os finalistas). Entrevistar todo mundo seria inviável; contratar só pela triagem seria arriscado. A combinação é barata e precisa.

Bônus: o contexto enviado ao LLM **encolhe** (de 6 para 3 trechos) e ao mesmo tempo **melhora** — menos tokens pagos, resposta mais focada.

Componente 3 — O experimento do tamanho do pedaço

Qual o tamanho ideal de cada pedaço da biblioteca? Em vez de chutar, a semana 5 testou três configurações com as mesmas perguntas:

Configuração	Relevância (1-5)	Qualidade (1-5)	O que se observou
250 caracteres	3	3	Fragmentação — cláusula de garantia cortada no meio da frase; o LLM reconstrói regras parciais
500 caracteres ← escolhido	5	5	Melhor equilíbrio : uma seção de política por pedaço, redundância mínima
1000 caracteres	4	4	Um pedaço longo domina a busca; a diversidade de contexto cai

A *lição de engenharia*: pedaço pequeno demais **quebra o sentido**; grande demais **dilui a busca**. O meio-termo (500) venceu — e agora essa escolha tem **evidência**, não opinião.

Componente 4 — As métricas: Precision@k e MRR

A parte mais importante da semana: **medir**. Duas métricas clássicas de busca, avaliadas sobre um conjunto de perguntas com gabarito (para cada pergunta, sabemos quais documentos são os certos):

- **Precision@k** ("precisão nos k primeiros"): dos k trechos que a busca trouxe, quantos eram certos? Se trouxe 3 e acertou 2, $P@3 = 0,67$.
- **MRR** (*Mean Reciprocal Rank*, "posição média do primeiro acerto"): o primeiro resultado certo veio em que posição? 1º lugar vale 1,0; 2º vale 0,5; 3º vale 0,33... Mede se **o melhor trecho chega no topo**.

```
# As funções são testadas com casos de gabarito conhecido antes de usar:
p3 = precision_at_k(["a", "b", "c"], relevant=["a", "c"], k=3)    # = 2/3 ✓
m  = mrr(["b", "a", "c"], relevant=["a"])                       # = 0.5 ✓ (acerto na 2ª posição)
```

E então o confronto — o mesmo conjunto de perguntas, os dois pipelines:

```
# Baseline (semana 4): busca por similaridade simples, k=4
# Otimizado (semana 5): MMR pesca 6 → cross-encoder fica com top-3

print(f"{'Pipeline':<35} {'P@3':>6} {'P@6':>6} {'MRR':>6}")
print(f"Baseline (similarity, k=4)          ...")
print(f"Optimised (MMR k=6 + rerank top-3)  ...")
# Requisito do Stop 2: MRR otimizado ≥ MRR baseline → ✓ PASSED
```

O notebook fecha com a tabela comparativa completa e o detalhamento pergunta a pergunta — provando com números que a otimização melhorou (ou pelo menos não piorou) cada métrica.

O toque final: citação obrigatória por afirmação

O prompt do pipeline otimizado ficou ainda mais exigente que o da semana 4 — agora **cada afirmação factual** precisa citar o arquivo de origem:

```
rag_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a TechStore Plus customer support assistant. "
     "Answer the question using ONLY the provided context. "
     "End every factual claim with the source filename in brackets, "
     "e.g. [policy_return_policy.txt].",          # ← cada frase factual sai com fonte!
    ("human", "Context:\n{context}\n\nQuestion: {question}"),
])
```

Resultado: respostas do tipo "A garantia padrão cobre 12 meses [policy_warranty.txt]. A estendida adiciona 1 ano [policy_extended_warranty.txt]." — auditáveis frase a frase.

Resumo da semana 5

Vício da semana 4	Remédio da semana 5
4 trechos repetidos do mesmo assunto	MMR (relevância + diversidade)
O melhor trecho nem sempre no topo	Cross-encoder re-ranking em 2 estágios
Tamanho de pedaço escolhido no chute	Experimento 250/500/1000 com evidência
"Parece melhor"	Precision@k e MRR, baseline × otimizado, com gabarito
Fontes só no rodapé	Citação por afirmação factual

A biblioteca agora é rápida, diversa, precisa e **medida**. Falta o último degrau: deixá-la à prova de produção — com trava contra respostas inventadas, capacidade de responder perguntas que cruzam vários documentos, e busca também em imagens. É a semana 6.

Passo a passo: como o código foi construído

O código da semana 5 mora no repositório do desafio M2 (c03-t05-bruno-pieri-m2-challenge/src/pipeline/), em três módulos: `vectorstore.py` (ganhou o MMR), `reranker.py` (novo) e `metrics.py` (novo). O notebook `Week5_RAG_Optimization.ipynb` os demonstra em sequência.

Passo 1 — O retriever MMR (`vectorstore.py::get_mmr_retriever`)

O código não só implementa — **documenta a matemática da decisão** no próprio docstring:

```
def get_mmr_retriever(vectorstore):
    """WHY MMR (not simple similarity)?
    Similarity search returns the top-k closest vectors. In a corpus where
    warranty terms appear in both policy_warranty_terms.txt and each
    product manual, all top-k results may be warranty chunks – the LLM
    receives redundant context and misses more specific product information.

    MMR selects the first document by pure similarity, then each subsequent
    document by the trade-off:
        lambda * sim(d, q) - (1 - lambda) * max_sim(d, selected)
    With lambda_mult=0.85, relevance is weighted more heavily than diversity
    while still reducing duplicate chunks. With fetch_k=20 >> k=6, MMR has
    enough candidates to find diverse results."""
```

Traduzindo a fórmula para português: *a nota de cada candidato = 85% de "quão relevante para a pergunta" – 15% de "quão parecido com o que eu já escolhi"*. Os três números calibrados:

Parâmetro	Valor	Significado
fetch_k	20	O lote inicial pescado por similaridade pura (matéria-prima para diversificar)
k	6	Quantos sobrevivem à seleção MMR
lambda_mult	0.85	O dial relevância ↔ diversidade (1.0 = só relevância; 0 = só diversidade)

Passo 2 — O re-ranker (`reranker.py`)

A escolha do modelo, documentada como constante:

```
_CROSS_ENCODER_MODEL = "cross-encoder/ms-marco-MiniLM-L-6-v2"
# Treinado no MS MARCO (pares pergunta-resposta, similar a suporte ao cliente).
# Rápido (6 camadas). Para mais precisão ao custo de latência:
# cross-encoder/ms-marco-electra-base.

RERANK_TOP_N = 3      # só 3 trechos chegam ao LLM – contexto enxuto e preciso

# Singleton de módulo: o modelo é carregado UMA vez, não a cada chamada
_cross_encoder = None
```

O padrão *singleton*: carregar um modelo de rede neural demora segundos; a variável de módulo `_cross_encoder = None` + O `if _cross_encoder is None` dentro da função garantem que o carregamento acontece na primeira chamada e nunca mais. (É o mesmo truque do `@lru_cache` do desafio M1 — outra grafia, mesma ideia.)

A função `rerank` — com validação de entrada e rastro nos metadados:

```
def rerank(query, docs, top_n=RERANK_TOP_N):
    if not docs:
        raise ValueError("docs must be non-empty")    # falha clara, não silêncio
    if top_n < 1:
        raise ValueError("top_n must be >= 1")

    # O cross-encoder pontua cada par (pergunta, trecho) LADO A LADO
    pairs = [(query, doc.page_content) for doc in docs]
    scores = _cross_encoder.predict(pairs)

    # Ordena por nota decrescente e fica com os top_n
    ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)[:top_n]

    # Cada trecho sai com a nota gravada nos metadados – transparência p/ depurar
    for doc, score in ranked:
        doc.metadata["rerank_score"] = float(score)
    return [doc for doc, _ in ranked]
```

O contrato não muda: entra lista de `Document`, sai lista de `Document` (menor e melhor). O resto do pipeline não precisa saber que o re-ranking existe — a peça é encaixável e removível.

Passo 3 — As métricas (`metrics.py`)

As duas funções centrais são pequenas o bastante para mostrar inteiras:

```
def precision_at_k(retrieved, relevant, k):
    """Dos top-k recuperados, que fração está no gabarito?"""
    if k <= 0:
        raise ValueError("k must be >= 1")
    hits = set(retrieved[:k]) & set(relevant)    # & = interseção de conjuntos
    return len(hits) / k
    # Exemplo: retrieved=[a,b,c], relevant=[a,c], k=3 → |{a,c}|/3 = 0.667

def mrr(retrieved, relevant):
    """1 / posição do primeiro acerto. Sem acerto → 0."""
    relevant_set = set(relevant)
    for i, doc_id in enumerate(retrieved):
        if doc_id in relevant_set:
            return 1.0 / (i + 1)    # 1º lugar → 1.0; 2º → 0.5; 3º → 0.33...
    return 0.0
```

O módulo traz também `EVAL_SET` (o conjunto de perguntas com gabarito — para cada pergunta, a lista dos arquivos que a respondem) e `evaluate_retriever` (roda qualquer retriever sobre o conjunto inteiro e agrega as médias).

Passo 4 — O notebook amarra tudo (na ordem do método científico)

1. **Setup** — ancora o diretório no repositório do desafio e carrega a estante existente (ou constrói, se for a primeira vez).

2. **Demonstração do MMR** — a consulta ampla de teste + a checagem de diversidade
(`unique_sources >= 3`).
3. **Demonstração do re-ranker** — antes × depois, com as notas visíveis.
4. **Experimento de chunk** — a tabela 250/500/1000 com observações qualitativas.
5. **Auto-teste das métricas** — antes de medir os pipelines, o notebook confere as próprias régua com casos de gabarito conhecido:

```
assert abs(precision_at_k(["a","b","c"], ["a","c"], k=3) - 2/3) < 1e-9
assert mrr(["b","a","c"], ["a"]) == 0.5
```

Isso é meta-rigor: medir com régua não aferida é pior que não medir. Três `assert` de uma linha eliminam a dúvida.

1. **O confronto final** — os dois pipelines definidos como funções intercambiáveis, avaliados sobre o mesmo `EVAL_SET` :

```
# Baseline: a semana 4 pura
baseline_retriever = vs.as_retriever(search_type="similarity", search_kwargs={"k": 4})

# Otimizado: o pipeline novo em uma função de duas linhas
def optimised_retriever_fn(query):
    mmr_docs = retriever.invoke(query)      # MMR pesca 6 diversos
    return rerank(query, mmr_docs)         # cross-encoder fica com top-3

# Mesma régua nos dois:
baseline_results, baseline_agg = evaluate_retriever(baseline_retriever.invoke, EVAL_SET)
optimised_results, optimised_agg = evaluate_retriever(optimised_retriever_fn, EVAL_SET)
```

1. **Tabela comparativa + veredito automatizado** — a última célula imprime P@3, P@6 e MRR lado a lado, calcula os deltas, e confere o requisito de aprovação do Stop 2 (`MRR otimizado ≥ MRR baseline`) imprimindo `✓ PASSED` ou `✗ FAILED` . O critério de sucesso não é uma opinião no relatório — é uma linha de código que qualquer pessoa pode re-executar.

Capítulo 7 · Semana 6 — A biblioteca à prova de produção (desafio M2)

Arquivo: `Week6_Stop3_Production_RAG.ipynb` (código no repositório `c03-t05-bruno-pieri-m2-challenge`) · **Entrega avaliada do módulo 2**

O que foi construído

A semana 6 é a entrega final do módulo 2 — e leva a biblioteca da semana 5 a nível de produção, adicionando três capacidades que sistemas RAG "de aula" não têm:

1. **Graph RAG** — um mapa de conexões entre entidades, para responder perguntas que **cruzam vários documentos**.
2. **Guardrails anti-alucinação** — um verificador que **confere cada afirmação** da resposta antes de entregá-la ao cliente.
3. **Busca multimodal** — além de texto, o sistema busca em **tabelas** (planilhas de especificações) e **imagens** (fotos e diagramas de produto).

Componente 1 — Graph RAG: o mapa de conexões

O problema que a busca vetorial não resolve

A busca por significado encontra trechos parecidos com a pergunta. Mas algumas perguntas exigem **seguir um caminho entre fatos que moram em documentos diferentes**: "*quais produtos são cobertos pela garantia estendida?*" — a resposta exige juntar o catálogo (produto A existe) com a política (o plano premium cobre a categoria X) com a tabela de planos (produto A está na categoria X). Nenhum trecho isolado contém a resposta inteira.

A solução: extrair um grafo de conhecimento

Durante o preparo da biblioteca, um LLM lê os documentos e extrai **triplas** — fatos mínimos no formato *sujeito → relação → objeto*:

Laptop Pro X1	--tem_garantia-->	Premium Protection Plan
Laptop Pro X1	--tem_memoria-->	16GB RAM
Premium Plan	--cobre-->	danos acidentais
Premium Plan	--dura-->	24 meses

Cada tripla guarda também **de onde veio** (documento fonte + a frase exata que a sustenta). As triplas viram um **grafo** — uma rede de bolinhas (entidades) e setas (relações):

```
kg = TechStoreKnowledgeGraph()
kg.extract_and_build(docs)      # LLM lê o corpus e extrai as triplas
print(f"Graph: {kg.graph.number_of_nodes()} nodes, {kg.graph.number_of_edges()} edges")

# Na hora da pergunta: partir de uma entidade e caminhar 1-2 "pulos" pelas setas
snippets = kg.query_subgraph(["Laptop Pro X1"], hops=2)
# → devolve a vizinhança: garantia do X1 → o que essa garantia cobre → por quanto tempo
```

💡 **Analogia** — é o quadro de investigação dos filmes policiais: fotos ligadas por barbantes. Perguntas simples se resolvem olhando uma foto ("qual o prazo de troca?"). Perguntas de investigação se resolvem **seguindo os barbantes**: da foto do laptop, dois barbantes adiante, chega-se à cobertura de danos acidentais. O `hops=2` significa "siga até dois barbantes de distância".

Componente 2 — Guardrails: o revisor que barra invenções

A peça mais crítica para pôr um RAG na frente de clientes. Mesmo com contexto bom, o LLM pode **alucinar** — misturar o que leu com o que "acha". A semana 6 monta uma linha de defesa em três etapas:

```
[1. ESCRITOR]      gera a resposta com uma citação [fonte] no fim de CADA frase
    ↓
[2. VERIFICADOR]   quebra a resposta em afirmações atômicas e confere cada uma
                  contra o trecho citado: o texto realmente sustenta isso?
    ↓
[3. PORTÃO]        decide o destino da resposta com base nas taxas de sustentação
```

```
raw = build_cited_answer(question, top_docs)      # escritor: resposta com citações
result = verify_answer(raw, top_docs)            # verificador + portão

print(result.decision)                          # a decisão do portão
print(result.claim_support_rate)                 # % das afirmações confirmadas pelo texto
print(result.contradiction_rate)                 # % que CONTRADIZ o texto (gravíssimo)
```

O portão tem **seis saídas possíveis**, em ordem decrescente de confiança:

Decisão	Quando	O cliente recebe
answer	Tudo confirmado	A resposta normal
answer_with_disclaimer	Quase tudo confirmado	A resposta + um aviso de incerteza
extractive	Confiança baixa na redação	Só os trechos literais dos documentos, sem parafrasear
no_answer	O acervo não cobre o assunto	"Não encontrei isso na nossa documentação"
ask_clarify	Pergunta ambígua	Um pedido de esclarecimento
refuse	Pergunta inadequada	Recusa educada

O notebook demonstra o portão funcionando com uma pergunta fora do escopo — *"qual é a capital da França?"* — que corretamente cai em `no_answer`: o sistema **prefere admitir ignorância a inventar**.

💡 **Analogia** — é o processo editorial de uma revista séria: o repórter (escritor) só publica frase com fonte; o **checador de fatos** (verificador) confere cada afirmação contra a fonte citada; e o editor (portão) decide — publica, publica com ressalva, corta para as aspas literais, ou não publica. Nenhuma frase chega ao leitor sem passar pelos três.

Componente 3 — Multimodal: texto + tabelas + imagens

Uma loja real tem informação em três formatos, e cada um pede um tratamento:

Modalidade	Acervo	Como é buscado	Etiqueta de citação
Texto	Manuais, políticas	Estante vetorial (MMR + re-rank, semana 5)	[arquivo.txt]
Tabelas	CSVs de especificações	Cada linha vira um documento buscável	[TB:...]
Imagens	Fotos e diagramas	Busca pela legenda descritiva de cada imagem	[I:...]

```
# Tabela: "quanto de RAM tem o Laptop Pro X1?" acha a LINHA certa do CSV
for doc in tr.retrieve("How much RAM does the Laptop Pro X1 have?"):
    print(doc.metadata['table_citation'], doc.page_content)

# Imagem: busca pela legenda descritiva
for doc in ir.retrieve("warranty tiers comparison figure"):
    print(doc.metadata['image_citation'], doc.page_content[:100])
```

E a **fusão tardia** (*late fusion*) junta tudo: cada modalidade é consultada **independentemente**, e os resultados são mesclados removendo duplicatas:

```
def late_fusion_retrieve(question, vs, tr, ir, ...):
    text_docs = rerank(question, retriever.invoke(question)) # modalidade texto
    table_docs = tr.retrieve(question, k=3) # modalidade tabela
    image_docs = ir.retrieve(question, k=2) # modalidade imagem

    # mescla, deduplicando pela fonte
    merged, seen = [], set()
    for doc in text_docs + table_docs + image_docs:
        if doc.metadata["source"] not in seen:
            seen.add(doc.metadata["source"])
            merged.append(doc)
    return merged
```

💡 **Analogia** — "tardia" porque cada especialista pesquisa **na sua própria seção** primeiro (o de texto na biblioteca, o de dados nas planilhas, o de imagens no acervo visual) e só **depois** as descobertas são postas na mesma mesa. A alternativa (converter tudo para um formato único antes) perderia a estrutura da tabela e o conteúdo visual.

A prova de fogo: 10 consultas difíceis + comparação com a semana 5

A entrega é validada com um conjunto de 10 perguntas **desenhadas para forçar cada capacidade nova**:

- *"Resuma a política de devolução citando pelo menos duas regras distintas"* (multi-citação)
- *"Quais produtos aparecem tanto na tabela de specs quanto no plano premium?"* (**cruzamento entre documentos** — Graph RAG)
- *"Quanto de RAM e armazenamento tem o Laptop Pro X1?"* (aterramento numérico — tabela)
- *"Identifique a imagem que mostra o Laptop Pro X1 e descreva o que ela mostra"* (imagem)
- *"A política permite reembolso após 30 dias?"* (armadilha temporal — a resposta certa é "não, o prazo é 7 dias")

E o rigor científico: as mesmas 10 perguntas rodam **também no pipeline da semana 5** (sem grafo, sem guardrails, sem multimodal), gerando um relatório comparativo com o que melhorou e — honestamente — o que regrediu (guardrails deixam o sistema mais conservador: ele responde "não sei" mais vezes, o que é o comportamento desejado em produção, mas derruba métricas ingênuas de "taxa de resposta").

O que o módulo 2 entregou, em uma frase

Uma biblioteca de conhecimento que busca por significado **e** por conexões **e** em três formatos, responde só com base em evidência citável, **confere cada frase antes de entregar**, e prefere admitir ignorância a inventar — tudo medido contra a versão anterior.

O módulo 3 muda o foco: das *fontes de conhecimento* para a *forma de raciocinar* — fluxos com várias etapas, especialistas e supervisão.

Passo a passo: como o código foi construído

O código do Stop 3 se distribui em três pastas novas do repositório M2 — `src/graph/`, `src/guardrails/`, `src/multimodal/` — construídas nessa ordem.

Passo 1 — O grafo de conhecimento (`graph/knowledge_graph.py`)

A **extração de triplas** usa o LLM com saída estruturada, mas com um filtro de qualidade:

```
def extract_and_build(self, documents):
    """Extraí triplas (sujeito, relação, objeto, citação) de cada documento
    via ChatOpenAI estruturado.

    Só entram no grafo triplas cuja relação está na DEFAULT_RELATION_ALLOWLIST –
    triplas de baixa confiança ou ruidosas são descartadas silenciosamente."""
```

A *allow-list de relações* é o *controle de qualidade*: o LLM extrai fatos livremente, mas só relações **conhecidas e úteis** (`tem_garantia`, `cobre`, `dura` ...) entram no mapa. Sem isso, o grafo viraria um emaranhado de relações inventadas e inconsistentes ("está_associado_a", "relaciona-se_com"...) que ninguém consegue percorrer com confiança.

A **consulta por vizinhança** — uma busca em largura com raio limitado:

```
def query_subgraph(self, seed_entities, hops=DEFAULT_HOPS, relation_allowlist=None):
    """A partir das entidades-semente, percorre o grafo até `hops` pulos.
    Cada resultado devolve:
        subject / relation / object    – o fato
        source_id                     – o ARQUIVO de onde o fato veio
        quote                         – a FRASE LITERAL que o sustenta
        hop                           – a distância da semente (1 ou 2)
    """
```

O *detalhe que muda tudo*: cada fato do grafo carrega **a frase literal de origem** (`quote`) e **o arquivo** (`source_id`). Quando o Graph RAG contribui para uma resposta, a contribuição é tão citável quanto um trecho da estante vetorial — o padrão de auditabilidade do projeto não abre exceções.

Passo 2 — O escritor com citação obrigatória (`guardrails/writer.py`)

```
def build_cited_answer(question, context_docs):
    """Gera a resposta com uma citação [arquivo_fonte] no FIM DE CADA FRASE.

    WHY CITATION BINDING?
    As citações servem a dois senhores: o usuário confere cada afirmação
    na fonte; e o verifier.py consegue checar cada claim olhando SÓ a
    fonte citada – O(claims) – em vez de comparar cada claim contra
    todos os chunks – O(claims × chunks)."""
```

O prompt do escritor impõe quatro regras: (1) usar SÓ o contexto; (2) terminar cada frase com `[arquivo]`; (3) se a resposta não estiver no contexto, dizer exatamente *"I don't have that information in our documentation."*; (4) nunca inventar números de modelo, preços ou datas.

Repare no argumento de eficiência do docstring: a citação por frase não é só transparência — ela **barateia a verificação**. O verificador sabe exatamente onde conferir cada afirmação, em vez de procurar em tudo. Design em que segurança e desempenho se reforçam.

Passo 3 — O verificador e o portão (`guardrails/verifier.py`)

O procedimento em três atos:

```
def verify_answer(answer, context):
    # 1. DECOMPOR: um LLM quebra a resposta em "claims atômicos"
    #   (uma afirmação factual por item, sem frases compostas)
    # 2. CONFERIR: para cada claim, outro prompt de "entailment" pergunta:
    #   o trecho citado SUSTENTA / CONTRADIZ / NÃO COBRE esta afirmação?
    # 3. DECIDIR: as taxas alimentam o portão
```

E o portão, com os limiares **documentados e justificados** no código:

```
SUPPORT_RATE_THRESHOLD = 0.85

# A lógica do portão (do docstring do módulo):
#   suporte >= 0.85 → decision="answer"
#   suporte < 0.85 E contradições == 0 → decision="answer_with_disclaimer"
#   contradições > 0 → decision="extractive"
#   (devolve o trecho LITERAL mais relevante)
#   sem evidência (contexto vazio/unknown) → decision="no_answer"

# WHY THESE THRESHOLDS?
#   0.85: tolera inferências contextuais menores, mas barra respostas onde
#         menos de 85% dos claims são verificáveis.
#   contradição > 0: UMA única contradição já dispara o fallback extrativo,
#   porque uma resposta que contradiz a fonte é ATIVAMENTE enganosa –
#   pior que uma resposta meramente não-verificada.
```

A *hierarquia moral embutida*: não-verificado ganha um aviso; **contraditório perde o direito de parafrasear** — o cliente recebe o texto literal do documento. O sistema distingue "não tenho certeza" de "estou dizendo o contrário da fonte", e trata o segundo como muito mais grave.

Passo 4 — Os retrievers multimodais (`multimodal/`)

Tabelas (`table_retriever.py`) — a sacada é a granularidade: cada **linha** do CSV vira um documento buscável independente ("Laptop Pro X1 | 16GB RAM | 512GB SSD" é um documento; a linha do outro laptop é outro). A pergunta "quanto de RAM tem o X1?" encontra exatamente a linha certa, com citação `[TB:laptop_specs:linha]`.

Imagens (`image_retriever.py`) — busca pela **legenda**: cada imagem do acervo tem uma descrição textual detalhada, e é a legenda que entra no índice. A pergunta "figura comparando os planos de garantia" casa com a legenda do diagrama certo, com citação `[I:nome_da_imagem]`. (Buscar pelo conteúdo visual em si exigiria embeddings de imagem — a legenda é a ponte pragmática.)

A fusão tardia (mostrada na seção anterior) junta os três com dois cuidados: cada modalidade falha **isoladamente** (um `try/except` por modalidade — se o índice de imagens quebrar, texto e tabelas continuam), e a deduplicação por fonte garante que a mesma evidência não entra duas vezes.

Passo 5 — A validação comparativa (o notebook)

A construção do experimento final tem um detalhe metodológico digno de nota: o notebook **reimplementa o pipeline da semana 5 em miniatura** (`stop2_answer` — MMR + rerank + LLM direto, sem guardrails, sem grafo, sem multimodal) para rodar as mesmas 10 perguntas nos dois sistemas **nas mesmas condições**:

```
def stop2_answer(question, vs):
    """Pipeline Stop 2 mínimo: MMR + rerank + LLM direto (sem guardrails)."""
    retriever = get_mmr_retriever(vs)
    mmr_docs = retriever.invoke(question)
    top_docs = rerank(question, mmr_docs)
    context = "\n---\n".join(d.page_content for d in top_docs)
    prompt = f"Answer the question using the context below.\n\nContext:\n{context}\n\nQuestion: {question}"
    ...
```

O relatório final compara os dois lado a lado e o notebook fecha com uma análise honesta em três seções: **o que melhorou** (cobertura de perguntas cruzadas, aterramento numérico, zero contradições não-sinalizadas), **limitações conhecidas** (o sistema ficou mais conservador — mais "não sei"; latência maior pelas checagens) e **próximos passos**. Entregar os trade-offs junto com as vitórias é o que diferencia um relatório de engenharia de um material de marketing.

PARTE III — Módulo 3: O Atendente que Raciocina em Etapas (LangGraph)

Capítulo 8 · Semana 7 — Abrindo a caixa-preta: o fluxo desenhado à mão

Arquivos: `Week7_LangGraph_Challenge.ipynb`, `src/chains/langgraph_challenge_agent.py`, `tests/test_langgraph_challenge_agent.py`

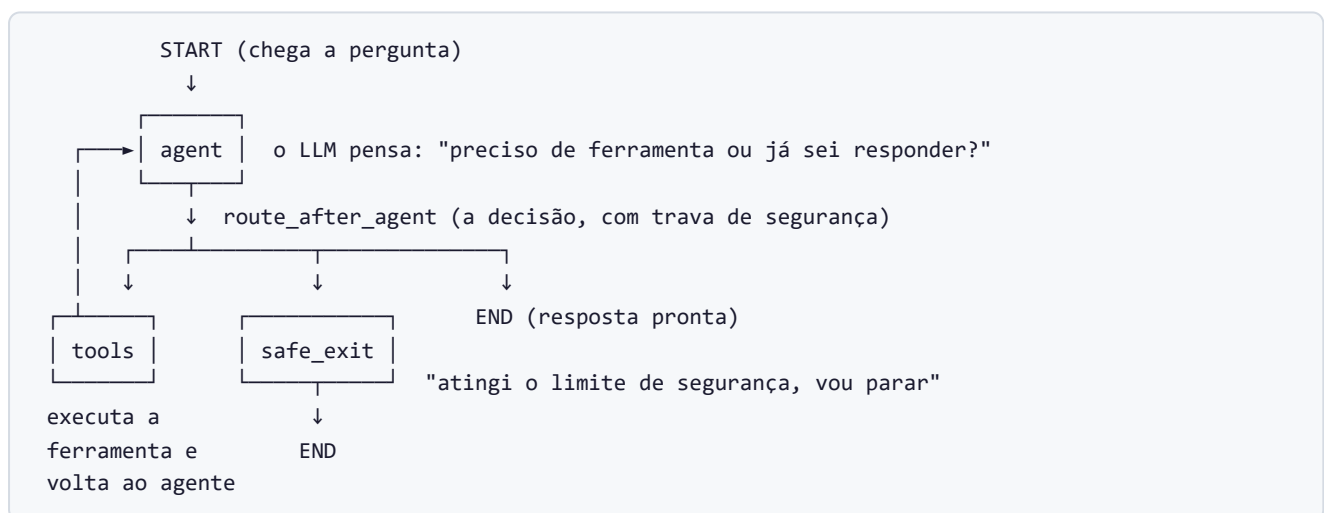
O que foi construído

Na semana 3, o agente usava uma peça pronta — `create_react_agent` — que escondia todo o ciclo "pensar → usar ferramenta → pensar" dentro de uma única chamada. Funcionava, mas era uma **caixa-preta**: impossível ver, ajustar ou testar o que acontecia lá dentro.

A semana 7 abre a caixa: o mesmo tipo de agente é **reconstruído à mão** com a biblioteca **LangGraph**, como um **grafo de estados** (*StateGraph*) — um fluxograma executável onde cada etapa, cada decisão e cada proteção fica explícita e testável.

O agente em si é proposital e humildemente simples — sabe somar e multiplicar. A simplicidade é estratégica: com ferramentas triviais, toda a atenção vai para **a estrutura do fluxo**, que é o que as semanas 8 e 9 vão sofisticar.

O fluxograma



💡 **Analogia** — a semana 3 comprou uma máquina de café de cápsula: aperta o botão, sai café, ninguém sabe como. A semana 7 monta a máquina peça por peça: moedor, aquecedor, válvulas — e agora dá para regular a moagem, trocar uma peça, e **instalar um desligamento automático** que a máquina de cápsula não tinha.

As peças, uma a uma

1. O estado tipado com "regras de acúmulo" (`AgentState`)

O **estado** é a prancheta compartilhada que circula entre as etapas do fluxo. Cada campo declara **como novas informações se combinam com as existentes** (o *reducer* — a regra de acúmulo):

```
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages] # falas: novas são ANEXADAS
    tool_calls: Annotated[int, operator.add] # contador: valores se SOMAM
    retries: Annotated[int, operator.add] # contador de re-tentativas
    errors: Annotated[list[str], operator.add] # lista de erros: concatena
```

Tradução: quando uma etapa devolve `{"tool_calls": 2}`, o LangGraph não *substitui* o contador — ele **soma** (regra `operator.add`). Isso elimina uma família inteira de bugs: nenhuma etapa consegue acidentalmente zerar o histórico ou o contador de outra. É a prancheta com regras de preenchimento impressas no cabeçalho.

2. Ferramentas que nunca explodem

Um detalhe de projeto fino: as ferramentas aceitam entradas "frouxas" (número **ou** texto) e **nunca lançam erro** — devolvem sempre uma resposta legível, mesmo para entrada inválida:

```
@tool
def add(a: str | float | int, b: str | float | int) -> str:
    """Add two numbers and return their sum."""
    parsed_a, error_a = _parse_number(a) # tenta converter para número
    if error_a:
        return f"ERROR:VALIDATION: invalid value for a - {error_a}"
        # ↑ devolve um erro EDUCADO E PADRONIZADO em vez de quebrar o programa
    ...
    return str(parsed_a + parsed_b)
```

Por quê? Se a ferramenta explodisse, o fluxo inteiro morreria no meio. Devolvendo o erro como texto padronizado (`ERROR:VALIDATION: ...`), **o agente fica sabendo do problema e pode reagir** — explicar ao usuário, tentar de novo, ou desistir com elegância. E o prefixo padronizado permite ao nó seguinte classificar o erro automaticamente: `VALIDATION` (fatal — não adianta repetir) ou `TRANSIENT` (passageiro — vale re-tentar).

 **Analogia** — é a diferença entre um funcionário que diante de um formulário rasurado **para tudo e vai embora**, e um que anota "campo X ilegível, favor reenviar" e segue o expediente. Sistemas de produção precisam do segundo.

3. O nó-agente: pensar e prestar contas

```
def agent_node(state):
    # 1. Confere os resultados das ferramentas da rodada anterior:
    #   algum devolveu "ERROR:..."? Registra na prancheta, classificado.
    for msg in reversed(state["messages"]):
        if not isinstance(msg, ToolMessage):
            break
        if content.startswith("ERROR:"):
            _, cls, detail = content.split(":", 2)    # separa a classe do erro
            new_errors.append(f"tool={msg.name} class={cls} ...")

    # 2. Pede ao LLM a próxima ação (responder? chamar ferramenta?)
    response = llm.invoke(state["messages"])

    # 3. Devolve o delta para a prancheta (os reducers acumulam)
    return {
        "messages": [response],
        "tool_calls": len(response.tool_calls),    # +N no contador
        "errors": new_errors,
        "retries": transient_count,
    }
```

4. A trava de segurança (`route_after_agent` + `safe_exit`)

O risco clássico de agentes: o **loop infinito** — o LLM fica pedindo ferramenta atrás de ferramenta, para sempre, queimando dinheiro. A proteção é uma **decisão condicional com limite**:

```
MAX_TOOL_CALLS = 5    # o disjuntor: no máximo 5 usos de ferramenta por rodada

def route_after_agent(state):
    base_route = tools_condition(state)    # o LLM pediu ferramenta?
    if base_route != "tools":
        return END                        # não pediu → conversa encerrada
    if state["tool_calls"] >= MAX_TOOL_CALLS:
        return "safe_exit"                # pediu, mas estourou o limite → saída segura
    return "tools"                        # pediu e há orçamento → executa
```

E a saída segura não é um erro seco — é uma **despedida honesta**:

```
def safe_exit_node(state):
    message = AIMessage(content=(
        "I've hit the tool-call safety limit (5) for this turn. "
        "I'm stopping here rather than looping indefinitely. "
        "Please rephrase your request more specifically. "))
    return {"messages": [message],
            "errors": [f"guard=loop_cap tool_calls={state['tool_calls']} limit=5"]}
```

💡 **Analogia** — é o disjuntor da casa. Ninguém espera precisar dele, mas quando algo entra em curto, ele **desarma antes do incêndio** — e deixa registrado no quadro qual circuito caiu.

5. Checkpoint e viagem no tempo (a semente da semana 9)

O grafo é compilado com um **checkpointer** — um gravador que fotografa a prancheta **a cada etapa**:

```
return workflow.compile(
    checkpointer=checkpointer,          # grava um snapshot do estado a cada passo
    interrupt_before=interrupt_before,  # opcional: pausar ANTES de um nó (ex.: ["tools"])
)
```

Isso habilita três superpoderes:

- **Retomar** — a conversa pode parar e continuar de onde estava (cada conversa tem um `thread_id`).
- **Pausar para aprovação** — `interrupt_before=["tools"]` congela o fluxo *antes* de executar uma ferramenta; alguém revisa e autoriza (a base do "humano no circuito" da semana 9).
- **Reexecutar (replay)** — voltar a qualquer fotografia e rodar de novo dali.

6. A fábrica testável (`build_graph`)

A função que monta tudo aceita **peças substituíveis**:

```
def build_graph(llm=None, checkpointer=None, interrupt_before=None):
    if llm is None:  # sem injeção → usa o LLM real da OpenAI
        llm = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(TOOLS)
    ...
```

Por que isso importa: os testes automatizados injetam um **LLM falso** (um dublê programado com respostas fixas) — e assim toda a estrutura do grafo (roteamento, trava, erros, checkpoint) é testada **sem gastar um centavo de API e sem depender da internet**. É o crash-test com boneco: ninguém arrisca um motorista para testar o airbag.

Os testes de aceitação

O notebook valida quatro cenários:

Teste	O que prova
Cadeia matemática — "some 2,5 e 7, depois multiplique por 3"	O agente encadeia duas ferramentas na ordem certa e responde 28,5
Trava de loop — um dublê malicioso pede ferramentas sem parar	O <code>safe_exit</code> desarma no 5º uso, com mensagem educada e erro registrado
Entrada inválida — "some 'abc' e 7"	A ferramenta devolve o erro padronizado; o agente explica sem quebrar
Replay — reexecutar a partir de um checkpoint	O histórico gravado permite voltar no tempo e continuar dali

Nota de rigor: o enunciado do desafio continha um erro aritmético — afirmava que $(2,5 + 7) \times 3 = 27$. O projeto detectou e validou o valor correto, **28,5**, documentando a discrepância. Conferir o gabarito também é engenharia.

Resumo da semana 7

Conceito novo	O que resolve
StateGraph explícito	O fluxo deixa de ser caixa-preta: visível, ajustável, testável
Estado com reducers	Etapas não sobrescrevem dados umas das outras
Ferramentas que não explodem	Erros viram informação útil, não crash
Trava de loop + saída segura	Nunca roda para sempre; falha com elegância
Checkpointing	Retomar, pausar para aprovação, viajar no tempo
Injeção de dublês	Estrutura 100% testada sem custo de API

Com a gramática do LangGraph dominada, as semanas 8 e 9 usam essas mesmas peças para montar algo muito maior: uma **equipe de agentes especialistas sob um supervisor**.

Passo a passo: como o código foi construído

O módulo `langgraph_challenge_agent.py` tem 214 linhas e foi escrito de dentro para fora: estado → ferramentas → nós → roteamento → fábrica. E os testes foram escritos **junto** — cada peça nasceu com seu teste.

Passo 1 — O estado (o contrato de dados de todo o fluxo)

```
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    tool_calls: Annotated[int, operator.add]
    retries: Annotated[int, operator.add]
    errors: Annotated[list[str], operator.add]
```

Como ler `Annotated[tipo, regra]`: o primeiro item é o tipo do campo, o segundo é a **regra de fusão** quando um nó devolve um valor novo. `add_messages` é a regra especializada do LangGraph para históricos de conversa (anexa e deduplica por id); `operator.add` soma números e concatena listas. Escolher a regra certa por campo é o design inteiro deste passo.

Passo 2 — As ferramentas (com o porquê no docstring do módulo)

O próprio arquivo documenta a decisão não-óbvia:

```
"""WHY loosely-typed tool arguments (str | float | int)
LangChain gera automaticamente um schema Pydantic a partir dos type hints
da função @tool. Se `a`/`b` fossem tipados como float, um valor ruim como
"abc" seria rejeitado por ESSE schema gerado ANTES do corpo da ferramenta
rodar — e a mensagem de erro seria do framework, não nossa.
Afrouxar os tipos deixa o corpo da ferramenta validar e produzir a
mensagem amigável."""
```

Tradução: existe um fiscal automático que rejeitaria a entrada errada **antes** da nossa função rodar — mas com uma mensagem técnica do framework. Afrouxando o tipo declarado, a entrada ruim **entra** na função, que a valida e devolve o erro educado padronizado (`ERROR:VALIDATION: ...`). O projeto prefere ser dono das próprias mensagens de erro.

E o auxiliar de conversão compartilhado pelas duas ferramentas:

```
def _parse_number(value):
    try:
        return float(value), None # sucesso: (número, sem erro)
    except (TypeError, ValueError):
        return None, f"could not parse {value!r} as a number" # falha: (nada, motivo)
```

O padrão de retorno duplo `(valor, erro)` evita exceções no caminho comum — quem chama decide o que fazer com o motivo da falha.

Passo 3 — O nó-agente como fábrica (`make_agent_node`)

Repare que `make_agent_node(11m)` é uma **função que devolve outra função**:

```
def make_agent_node(llm):
    def agent_node(state):
        ...
        response = llm.invoke(state["messages"])
        ...
    return agent_node
```

Por quê? Nós do LangGraph recebemos só o estado — não têm onde receber o LLM. A fábrica "embala" o LLM dentro do nó no momento da construção (o padrão *closure*). É essa embalagem que permite aos testes injetarem o dublê.

Dentro do nó, o **varredor de erros** examina os resultados de ferramentas da rodada anterior:

```
for msg in reversed(state["messages"]):
    if not isinstance(msg, ToolMessage):
        break
    if content.startswith("ERROR:"):
        _, cls, detail = content.split(":", 2) # "ERROR:VALIDATION: detalhe" → 3 partes
        new_errors.append(f"tool={msg.name} class={cls} detail={detail.strip()[:160]}")
```

Por que varrer de trás para frente é seguro: o comentário no código explica — a única aresta que chega ao agente (além do START) é `tools → agent`, então os resultados de ferramenta recém-produzidos são sempre o bloco final contíguo das mensagens. Entender a topologia do próprio grafo permite código mais simples.

Passo 4 — O roteamento com a trava (e uma sutileza de timing)

```
def route_after_agent(state):
    """O loop-cap vive AQUI (na aresta condicional), não dentro do nó —
    nós só devolvemos deltas de estado; só arestas escolhem o próximo passo.

    O contador tool_calls foi incrementado pelo agent_node ANTES desta função
    rodar (o LangGraph aplica o delta do nó antes de avaliar a aresta) —
    então um pedido acima do limite é roteado para safe_exit e ABANDONADO
    antes de ser executado."""
```

A sutileza: a ferramenta número 5 **nunca roda**. O contador conta *pedidos*, o delta é aplicado antes da decisão, e o pedido que estouraria o limite morre na triagem. O teste de loop confirma isso com precisão cirúrgica (abaixo).

Passo 5 — A fábrica do grafo (`build_graph`)

```
workflow = StateGraph(AgentState)           # 1. a prancheta define o grafo
workflow.add_node("agent", make_agent_node(llm)) # 2. os três nós
workflow.add_node("tools", ToolNode(TOOLS))    # (ToolNode: executor pronto do LangGraph)
workflow.add_node("safe_exit", safe_exit_node)

workflow.add_edge(START, "agent")             # 3. as arestas fixas
workflow.add_conditional_edges(                # 4. a aresta condicional com o MAPA
    "agent", route_after_agent,
    {"tools": "tools", "safe_exit": "safe_exit", END: END},
)                                              # (o que a função devolve → para onde ir)
workflow.add_edge("tools", "agent")           # 5. o ciclo: ferramenta volta ao agente
workflow.add_edge("safe_exit", END)

return workflow.compile(checkpointer=checkpointer,      # 6. compila com gravador
                        interrupt_before=interrupt_before) # e pausas opcionais
```

Seis passos declarativos — o fluxograma do início do capítulo, linha por linha.

Passo 6 — Os testes: o dublê e as provas

O dublê de LLM cabe em 10 linhas — e é a chave de todo o rigor da semana:

```
class FakeToolCallingLLM:
    """LLM falso: devolve respostas pré-programadas, uma por chamada."""
    def __init__(self, responses):
        self._responses = list(responses)    # o roteiro
        self.calls = []                     # o registro (para inspeção)

    def invoke(self, messages):
        self.calls.append(messages)          # anota o que recebeu
        return self._responses.pop(0)        # devolve a próxima fala do roteiro
```

Teste 1 — a cadeia matemática (com a correção do gabarito documentada):


```
def test_math_chain_uses_both_tools_and_returns_28_5():
    # NOTA: o exemplo do enunciado ("some 2,5 e 7, multiplique por 3" → 27.0)
    # é aritmeticamente inconsistente: (2.5 + 7) * 3 = 28.5. Este teste valida
    # o resultado CORRETO em vez do número literal (errado) do enunciado.
    fake_llm = FakeToolCallingLLM([
        _tool_call_message("add", {"a": 2.5, "b": 7}), # roteiro: pede a soma
        _tool_call_message("multiply", {"a": 9.5, "b": 3}), # depois a multiplicação
        AIMessage(content="2.5 + 7 = 9.5, then 9.5 * 3 = 28.5."), # e conclui
    ])
    app = build_graph(llm=fake_llm, checkpointer=InMemorySaver())
    result = app.invoke(initial_state("Add 2.5 and 7, then multiply by 3."), ...)

    assert "28.5" in result["messages"][-1].content
    assert result["tool_calls"] == 2 # exatamente 2 ferramentas
    assert result["errors"] == [] # zero erros
    assert len(fake_llm.calls) == 3 # o LLM foi consultado exatamente 3 vezes
```

Teste 2 — a trava, com um dublê sabotador que **nunca** desiste:

```
def test_loop_cap_exits_via_safe_exit_at_exactly_five():
    # O roteiro só tem pedidos de ferramenta – nunca uma resposta final
    responses = [_tool_call_message("add", {"a": 1, "b": 1})
                 for _ in range(MAX_TOOL_CALLS + 2)]

    ...
    assert result["tool_calls"] == MAX_TOOL_CALLS # parou EXATAMENTE em 5
    assert "safety limit" in result["messages"][-1].content # a despedida educada saiu
    assert any(e.startswith("guard=loop_cap") for e in result["errors"]) # e ficou registrado
    assert len(fake_llm.calls) == MAX_TOOL_CALLS # o pedido que estouraria nunca executou
```

A última asserção é a prova da sutileza do passo 4: o dublê tinha 7 respostas no roteiro, mas só foi consultado 5 vezes — a triagem abandonou o excesso antes de executar. Testes bons não conferem só o resultado; conferem o **mecanismo**.

Capítulo 9 · Semana 8 — Do agente único à central de especialistas (M3, stops 1 e 2)

Repositório: `c03-t05-bruno-pieri-m3-challenge` · **Arquivos:** `challenge.ipynb`, `src/graph/stop1_agent.py`, `src/graph/stop2_agent.py`, `src/database/mock_db.py`

O contexto: o desafio final (M3) começa

O módulo 3 fecha o curso com um desafio em três paradas (*stops*), no mesmo modelo dos anteriores: as duas primeiras são formativas (exercícios de construção) e a terceira é a entrega avaliada. Todas usam a TechStore Plus como cenário e um **banco de dados simulado** fornecido pronto (`mock_db.py` — clientes, pedidos, garantias, chamados).

- **Stop 1** — reconstruir o agente básico, agora com ferramentas de loja de verdade.
- **Stop 2** — reorganizá-lo numa **central de triagem**: um roteador + três especialistas.
- **Stop 3** (capítulo 10) — promover tudo a uma equipe supervisionada, com seleção dinâmica de ferramentas e aprovação humana.

Stop 1 — O agente básico, versão TechStore

O stop 1 é a semana 7 aplicada ao domínio da loja: mesma anatomia (estado tipado, ToolNode, trava de loop, checkpointing), mas as ferramentas deixam de ser `add/multiply` e passam a consultar o banco da loja:

```
@tool
def customer_lookup(email: str) -> str:
    """Look up a customer's profile and loyalty tier by email address."""
    customer = db.get_customer(email)
    if customer is None:
        return f"No customer found for email {email!r}." # nunca explode: responde educado
    return (f"{customer['name']} ({customer['id']}) - tier: {customer['tier']}, "
            f"status: {customer['status']], region: {customer['preferred_region']}.")

@tool
def order_status(order_id: str) -> str:
    """Look up an order's current status, tracking number, and delivery estimate."""
    order = db.get_order(order_id)
    ...
```

O estado ficou até mais enxuto que o da semana 7 — sinal de fluência na ferramenta:

```
class Stop1State(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages] # o histórico
    tool_calls: Annotated[int, operator.add] # o contador da trava
```

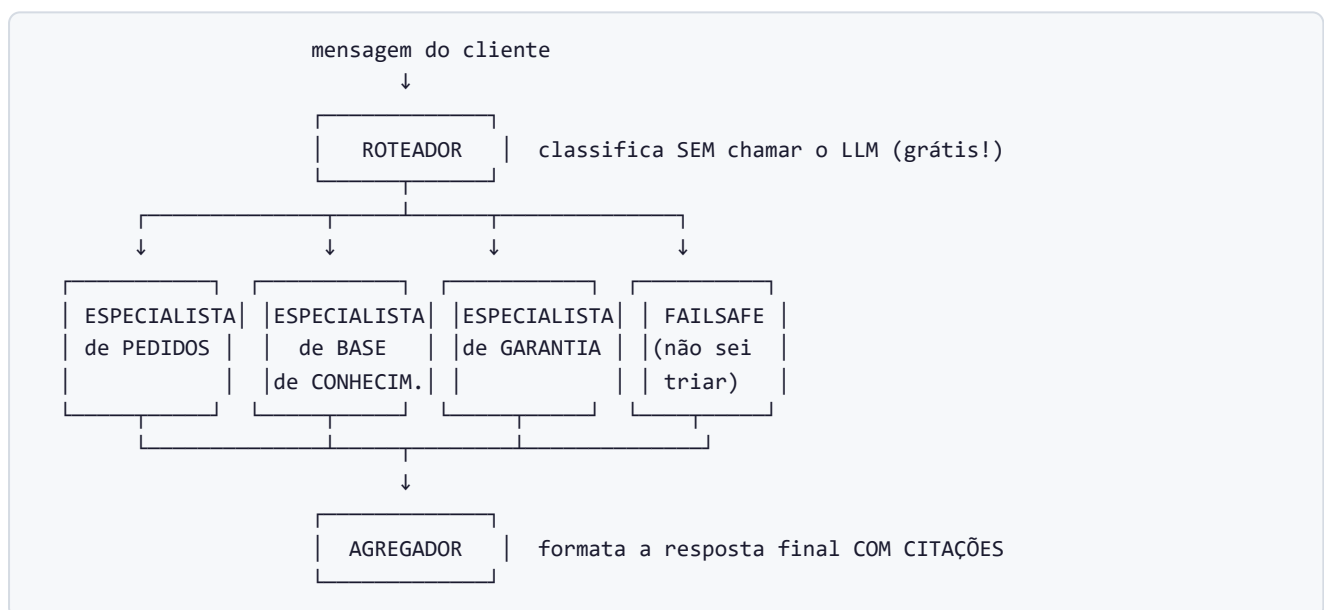
Nada aqui é novidade conceitual — é o **kata de consolidação**: repetir a estrutura até ela ficar automática, porque os stops seguintes vão empilhar complexidade em cima dela.

Stop 2 — A central de triagem: roteador → especialistas → agregador

Aqui está a mudança arquitetural da semana. Um agente único com todas as ferramentas funciona, mas tem defeitos que crescem com a escala:

- **Ferramenta errada na mão errada** — com 20 ferramentas disponíveis, o LLM às vezes escolhe mal.
- **Prompt genérico** — um faz-tudo não tem a profundidade de um especialista.
- **Sem controle de acesso** — qualquer pergunta pode acionar qualquer ferramenta.

A solução espelha um call center real:



O roteador que não gasta um centavo

Detalhe de engenharia elegante: a triagem **não usa LLM** — é uma heurística de palavras-chave, instantânea e gratuita:

```
def router_node(state):
    """Classifica em order / kb / warranty / failsafe. A ordem dos testes importa:
    'garantia do pedido ORD-002' deve cair em WARRANTY, não em ORDER —
    a palavra 'warrant' é mais específica que 'order'."""
    text = _latest_query(state).lower()
    if "warrant" in text:
        route = "warranty" # garantia vence (mais específica)
    elif re.search(r"\bord-\d+\b", text) or any(kw in text for kw in ("order", "tracking", ...)):
        route = "order" # menciona pedido/rastreo
    elif any(kw in text for kw in ("polic", "return", "product", "faq")):
        route = "kb" # pergunta de política/produto
    else:
        route = "failsafe" # não sei triar → resposta segura
    return {"route": route}
```

Lição repetida do curso: **não use IA onde uma regra resolve.** A triagem de 4 categorias com palavras-chave óbvias não precisa de um LLM — e a ordem dos testes (do mais específico ao mais genérico) resolve as ambiguidades.

💡 **Analogia** — é a atendente da recepção que ouve 5 segundos e já transfere o ramal — sem consultar ninguém. E repare no **failsafe**: quando ela não sabe para onde transferir, existe um protocolo educado em vez de uma transferência aleatória.

Especialistas com crachá de acesso restrito

Cada especialista recebe **apenas as suas ferramentas** — uma *allow-list* (lista de permissões):

```
ORDER_TOOLS    = [lookup_order_s2, lookup_customer_s2] # o de pedidos consulta pedidos e cadastro
KB_TOOLS       = [search_kb_s2]                       # o de conhecimento só busca artigos
WARRANTY_TOOLS = [warranty_days_s2]                   # o de garantia só calcula prazos
```

Por que isso importa (segurança e qualidade): o especialista de base de conhecimento **fisicamente não consegue** consultar dados de pedidos de clientes — errar de ferramenta virou impossível, não improvável. Cada um também tem seu prompt focado e um limite de **uma chamada de ferramenta por turno** (disciplina que evita vagoio).

Os nomes levam o sufixo `_s2` de propósito: os três stops mantêm **espaços de ferramentas separados** (stop 1, stop 2, stop 3), sem compartilhamento acidental — higiene de código em projeto com fases.

O agregador e as citações

A última etapa formata a resposta final e **anexa citações** de onde cada informação veio — a mesma disciplina de auditabilidade do módulo 2, agora no mundo dos agentes:

```
class Stop2State(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    route: str # decidido uma vez pelo roteador
    citations: Annotated[list[str], operator.add] # acumula: ["order:ORD-001",
"kb:return_policy"]
```

Cada uso de ferramenta gera uma etiqueta (`order:ORD-001` , `kb:return_policy`) que o agregador junta à resposta. Quem lê sabe exatamente **qual consulta sustentou qual afirmação**.

A mini base de conhecimento

O especialista de conhecimento busca em quatro artigos internos (política de devolução, garantia, envio, catálogo) com um casamento simples de palavras-chave — um RAG de brinquedo, suficiente para exercitar o fluxo. O projeto sabe onde mora a versão séria disso: no módulo 2.

O que os dois stops entregaram

	Stop 1	Stop 2
Estrutura	1 agente + ferramentas	roteador → 3 especialistas + failsafe → agregador
Triagem	o LLM decide tudo	heurística gratuita, sem LLM
Ferramentas	todas no mesmo bolso	allow-list por especialista
Resposta	texto do LLM	texto + citações das consultas
Proteções	trava de loop, checkpoint	idem + rota failsafe

Mas os especialistas do stop 2 ainda são **funções simples** dentro do grafo principal — todos dividem a mesma prancheta, e a triagem por palavra-chave tem seus limites. O stop 3 promove cada especialista a um **subgrafo independente e compilado**, coloca um **supervisor** de verdade no comando, adiciona **seleção dinâmica de ferramentas** e um **portão de aprovação humana**. É o capítulo final.

Passo a passo: como o código foi construído

O stop 2 (`src/graph/stop2_agent.py` , 300 linhas) é o mais instrutivo para dissecar, porque mostra a **transição** — o momento em que a arquitetura muda de forma. Vamos na ordem do arquivo.

Passo 1 — O estado com dois estilos de campo

```
class Stop2State(TypedDict):
    """`route` usa o reducer padrão de SOBRESCRITA (o roteador o define uma vez);
    `citations` ACUMULA entre turnos de ferramenta via operator.add."""
    messages: Annotated[list[BaseMessage], add_messages]
    route: str # sem Annotated = sobrescreve
    citations: Annotated[list[str], operator.add] # com operator.add = acumula
```

A escolha por campo é deliberada: a rota é uma decisão única (o valor novo substitui), as citações são um diário (valores novos se somam). Errar essa escolha causaria bugs sutis — uma rota que "acumula" viraria lixo; citações que "sobrescrevem" perderiam histórico.

Passo 2 — As ferramentas com namespace próprio

As quatro ferramentas do stop 2 (`lookup_order_s2`, `lookup_customer_s2`, `search_kb_s2`, `warranty_days_s2`) seguem o gabarito de sempre (docstring-vitrine, nunca explodem, normalizam entrada). A busca da mini base de conhecimento merece uma olhada — um "RAG de 10 linhas":

```
def _kb_search(query):
    """Casamento ingênuo: conta palavras do slug presentes na pergunta."""
    query_lower = query.lower()
    best_slug, best_score = None, 0
    for slug, content in KB_ARTICLES.items(): # ex.: slug = "return_policy"
        score = sum(1 for keyword in slug.split("_") # palavras: "return", "policy"
                    if keyword in query_lower) # quantas aparecem na pergunta?
        if score > best_score:
            best_slug, best_score = slug, score # guarda o melhor
    return (best_slug, KB_ARTICLES[best_slug]) if best_slug else (None, None)
```

Passo 3 — O coração: `_run_specialist` (o turno de um especialista)

A função genérica que **todos** os especialistas compartilham — só mudam as ferramentas e o domínio:

```

def _run_specialist(state, tools, domain, llm):
    tools_by_name = {t.name: t for t in tools}          # o catálogo DESTA especialista
    response = llm.invoke(state["messages"])            # 1º pensamento do LLM

    if not getattr(response, "tool_calls", None):
        return {"messages": [response]}                # não pediu ferramenta? já respondeu.

    if len(response.tool_calls) > 1:                    # pediu VÁRIAS? disciplina:
        logger.warning("%s specialist requested %d tool calls; truncating to the first.",
                        domain, len(response.tool_calls))
        response = response.model_copy(update={"tool_calls": response.tool_calls[:1]})
        # ↑ corta para UMA, registrando o desvio no log (visibilidade sem drama)

    call = response.tool_calls[0]
    tool_fn = tools_by_name[call["name"]]               # só encontra ferramentas do SEU bolso
    result = tool_fn.invoke(call["args"])               # executa
    tool_message = ToolMessage(content=result, name=call["name"], tool_call_id=call["id"])

    citation = _citation_for(call["name"], call["args"], result, domain) # gera a etiqueta
    follow_up = llm.invoke(state["messages"] + [response, tool_message]) # 2º pensamento:
                                                # redigir a resposta COM o resultado

    return {
        "messages": [response, tool_message, follow_up],
        "citations": [citation] if citation else [],    # a etiqueta vai para o diário
    }

```

O ritmo de um turno: pensar → (no máximo uma) ferramenta → pensar de novo com o resultado → responder. A regra de "uma ferramenta por turno" não é limitação técnica — é **disciplina imposta**, com log de aviso quando o LLM tenta exagerar.

Passo 4 — A fábrica de especialistas com carregamento preguiçoso

```

def _build_specialist_node(tools, domain, llm=None):
    def node(state):
        # Construído PREGUIÇOSAMENTE: especialistas não-roteados nunca criam
        # um ChatOpenAI real (e nunca precisam de OPENAI_API_KEY) numa execução.
        active_llm = llm or ChatOpenAI(model=MODEL, temperature=0).bind_tools(tools)
        return _run_specialist(state, tools, domain, active_llm)
    return node

```

Dois ganhos numa tacada: (1) uma pergunta de garantia **nunca** cria os LLMs de pedidos e conhecimento — não paga o custo do que não usa; (2) o parâmetro `llm=None` é a porta dos testes — o dublê entra por aqui, especialista por especialista (`llms={"order": fake, ...}` na fábrica do grafo).

Passo 5 — Agregador e failsafe (os dois finais possíveis)

```
def aggregator_node(state):
    """Anexa as etiquetas de citação acumuladas à resposta final."""
    last = state["messages"][-1]
    tags = " ".join(f"[{c}]" for c in state.get("citations", []))
    content = f"{last.content} {tags}".strip() if tags else last.content
    return {"messages": [AIMessage(content=content)]}
    # resultado: "Seu pedido chega dia 20. [order:ORD-001]"

def failsafe_node(state):
    """Entrada ambígua → pede esclarecimento e vai DIRETO para END
    (não passa pelo agregador — não há consulta a citar)."""
    return {"messages": [AIMessage(content=
        "I'm not sure how to help with that — could you clarify whether this is "
        "about an order, a warranty, or a general product question?"))]}
```

Repare que até a topologia comunica: o failsafe liga direto no END, sem agregador — porque não tem citação a anexar. O desenho do grafo reflete o significado.

Passo 6 — A montagem do grafo

```
workflow = StateGraph(Stop2State)
workflow.add_node("router", router_node)
workflow.add_node("order_specialist", _build_specialist_node(ORDER_TOOLS, "order", ...))
workflow.add_node("kb_specialist", _build_specialist_node(KB_TOOLS, "kb", ...))
workflow.add_node("warranty_specialist", _build_specialist_node(WARRANTY_TOOLS, "warranty", ...))
workflow.add_node("aggregator", aggregator_node)
workflow.add_node("failsafe", failsafe_node)

workflow.add_edge(START, "router")
workflow.add_conditional_edges(
    "router",
    lambda state: state["route"],          # a decisão é simplesmente LER o campo route
    {"order": "order_specialist", "kb": "kb_specialist",
     "warranty": "warranty_specialist", "failsafe": "failsafe"},
)
workflow.add_edge("order_specialist", "aggregator")    # os 3 especialistas convergem
workflow.add_edge("kb_specialist", "aggregator")
workflow.add_edge("warranty_specialist", "aggregator")
workflow.add_edge("aggregator", END)
workflow.add_edge("failsafe", END)                  # o failsafe sai direto
```

Um refinamento em relação à semana 7: a aresta condicional agora é um `lambda` trivial que só lê `state["route"]` — porque a decisão **já foi tomada e gravada** pelo roteador. Separar "decidir" (nó) de "encaminhar" (aresta) deixa a decisão testável isoladamente: o teste do roteador não precisa rodar o grafo.

Capítulo 10 · Semana 9 — A equipe supervisionada (M3, stop 3 — entrega avaliada)

Repositório: `c03-t05-bruno-pieri-m3-challenge` · **Arquivos:** `src/graph/specialists.py` , `src/graph/supervisor.py` , `src/selector/bigtool.py` , `src/timetravel/repair.py` , `tests/test_stop3.py`

O que foi construído

A entrega final do curso reúne tudo num sistema multi-agente de nível profissional, com **quatro capacidades** que respondem a quatro problemas reais de produção:

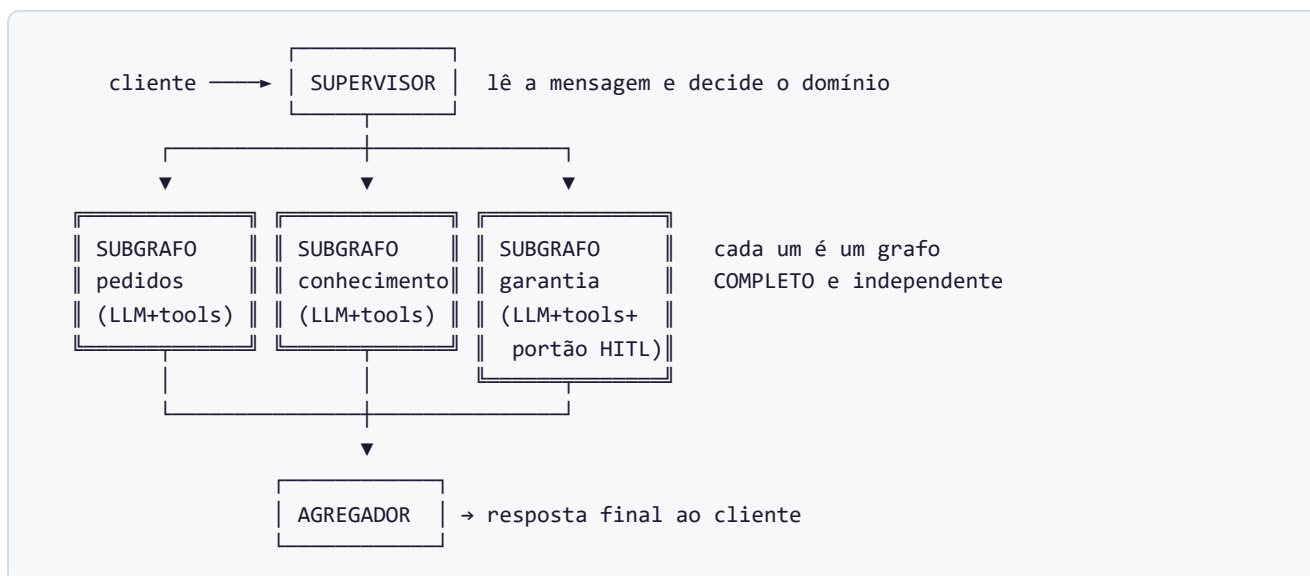
Problema de produção	Solução do stop 3
Um agente só não escala para muitos domínios	Supervisor coordenando especialistas independentes
LLMs se atrapalham com muitas ferramentas	BigTool : seleção dinâmica das 4 melhores por pedido
Ações de risco não podem ser 100% automáticas	HITL : pausa para aprovação humana antes de criar chamado
Quando algo dá errado, refazer tudo é caro	Time-travel : editar um checkpoint e retomar dali

Peça 1 — O supervisor e os subgrafos (hub-and-spoke)

A evolução em relação ao stop 2 é estrutural: cada especialista deixa de ser uma função solta e vira um **subgrafo compilado** — um grafo completo e independente (com seu próprio estado, ferramentas e proteções) que é **plugado como uma peça** dentro do grafo do supervisor:

```
# COMPILA os grafos internos primeiro...
order    = build_order_specialist()    # grafo completo do especialista de pedidos
kb       = build_kb_specialist()      # ...da base de conhecimento
warranty = build_warranty_specialist() # ...de garantia (com o portão HITL dentro)

# ...DEPOIS pluga cada um como um nó do grafo do supervisor
builder.add_node("order", order)
builder.add_node("kb", kb)
builder.add_node("warranty", warranty)
```



💡 **Analogia** — o stop 2 era um escritório com três funcionários na mesma sala, dividindo a mesma prancheta. O stop 3 é uma empresa com **três departamentos**, cada um com sua sala, seus arquivos e seus processos internos — e um **gerente geral** (supervisor) que recebe o cliente, decide qual departamento resolve, e um **secretário** (agregador) que redige a resposta final. Departamentos podem ser testados, trocados ou promovidos sem mexer nos outros.

Um refinamento técnico que virou correção de bug real: as ferramentas do catálogo eram funções anônimas sem descrição, e tanto o executor quanto o LLM **precisam da descrição** para montar o esquema da ferramenta. A solução (`_resolve_tools`) embrulha cada entrada do catálogo num `StructuredTool` com nome, descrição (`use_for`) e esquema de argumentos vindos do próprio catálogo — sem isso, o grafo nem compilava.

Peça 2 — BigTool: a caixa de ferramentas sob demanda

O problema

Um sistema maduro acumula dezenas de ferramentas. Mas LLMs **degradam com mais de ~5 ferramentas à vista**: a latência sobe e os erros de escolha se multiplicam. Dar o catálogo inteiro ao agente é como despejar 50 chaves na bancada do mecânico.

A solução: filtrar por política, ranquear, entregar no máximo 4

```
RISK_ORDER = {"low": 0, "med": 1, "high": 2}    # escala de risco das ferramentas

def select(self, request, ctx):
    # Passo 1 – FILTRO DE POLÍTICA (portão duro, não desempate):
    candidates = [s for s in CATALOG if self._passes_policy(s, ctx)]

    # Passo 2 – RANQUEAR por aderência ao pedido:
    tokens = set(request.lower().split())
    def score(spec):
        overlap = len(tokens & set(spec.use_for.lower().split())) # palavras em comum
        return (overlap, -RISK_ORDER[spec.risk]) # empate? vence a mais SEGURA
    candidates.sort(key=score, reverse=True)

    # Passo 3 – entregar NO MÁXIMO 4:
    return candidates[:4]
```

E o filtro de política — quatro portões que a ferramenta precisa atravessar:

```
def _passes_policy(self, spec, ctx):
    if not spec.healthy:                                # serviço fora do ar? barrada.
        return False
    if spec.region != "global" and spec.region != ctx.region: # região errada? barrada.
        return False
    if not (spec.scopes & ctx.scopes):                    # sem permissão? barrada.
        return False
    if RISK_ORDER[spec.risk] > RISK_ORDER[ctx.risk_ceiling]: # risco acima do teto? barrada.
        return False
    return True
```

O detalhe de projeto que o módulo enfatiza: **a política filtra ANTES do ranking** — é um portão, não um critério de desempate. Se fosse depois, uma ferramenta perigosa mas "muito relevante" poderia passar na frente. Segurança não disputa pontos com conveniência.



Analogia — é o almoxarifado de um hospital: o funcionário não leva o depósito inteiro ao centro cirúrgico. Ele confere **quem** está pedindo e **que autorização tem** (política), separa o que serve **para aquele procedimento** (ranking) e leva **uma bandeja pequena** (top-4). E material vencido ou interditado nem entra na triagem.

Peça 3 — HITL: o humano no circuito

HITL (*Human-In-The-Loop*) é a resposta à pergunta: *o que um agente autônomo NÃO deve fazer sozinho?* Aqui, a ação sensível é **criar um chamado de garantia** (`ticket_create`) — algo que dispara processos reais na empresa. Antes de executá-la, o grafo **congela e espera um humano**:

```
# Dentro do especialista de garantia, no nó que antecede a criação do chamado:
interrupt("ticket_create requires operator approval. Resume to proceed.")
# ↑ o grafo INTEIRO suspende aqui. O estado fica gravado no checkpoint.
# Um operador humano inspeciona e, se aprovar, o fluxo RETOMA do ponto exato.
```

Um detalhe arquitetural deliberado: o portão vive **dentro do subgrafo de garantia**, não no supervisor. A regra de aprovação pertence a quem executa a ação — se um dia outro caminho chegar à mesma ação, o portão continua lá, impossível de contornar.

💡 **Analogia** — o caixa do banco resolve consultas de saldo sozinho, mas transferências acima de um valor **travam no sistema até a assinatura do gerente**. O sistema não confia na boa vontade do caixa: a trava está embutida na própria operação.

É a colheita do que a semana 7 plantou: o `interrupt` só é possível porque **cada passo do grafo é checkpointado** — dá para congelar, guardar e retomar sem perder nada.

Peça 4 — Time-travel: consertar o passado sem refazer tudo

A capacidade mais surpreendente. Como cada transição de estado vira um **checkpoint** (fotografia), é possível — quando algo deu errado no meio de uma execução — **voltar à fotografia problemática, corrigi-la, e retomar dali**, sem re-executar tudo do zero:

```
def repair_run(app, config, edit, as_node):
    # 1. Lista o histórico de fotografias (mais recente primeiro)
    history = list(app.get_state_history(config))
    before_state = dict(history[0].values)          # 2. guarda o "antes"

    # 3. Aplica a correção NO checkpoint — isso cria um RAMO NOVO na árvore;
    #    o ramo original defeituoso é PRESERVADO (auditoria!)
    resumed_cfg = app.update_state(target_cfg, edit, as_node=as_node)

    # 4. Retoma a execução a partir do checkpoint corrigido
    app.invoke(None, resumed_cfg)                  # None = "continue de onde parou"

    # 5. Devolve o diff: antes x depois x id do novo ramo
    return {"before": before_state, "after": after_state, "branch_id": branch_id}
```

Quando usar cada abordagem (critério documentado no próprio código):

Situação	Abordagem	Custo
A entrada estava errada (e-mail com typo, pedido errado)	Re-executar do zero	Alto — o grafo inteiro roda de novo
O fluxo rodou certo mas um dado no meio estava velho/errado	Time-travel: editar o checkpoint e retomar	Baixo — só o trecho após a correção roda

💡 **Analogia** — um documento com controle de versões. Descobriu-se um número errado na página 12 de um relatório de 200 páginas: ninguém redige o relatório inteiro de novo — volta-se à versão da página 12, corrige-se o número, e o restante é regenerado a partir dali. E a versão errada **fica no histórico**, porque auditoria importa: o novo ramo não apaga o antigo.

A validação: os três casos obrigatórios

A entrega é aprovada por três testes automatizados (`tests/test_stop3.py`) — os mesmos que este projeto validou com a API real:

Caso	O que prova	Precisa de LLM real?
A — Roteamento do supervisor	Uma conversa com perguntas de domínios diferentes aciona ≥ 2 especialistas distintos	Sim
B — Política e top-K do BigTool	Ferramentas doentes/sem permissão/de risco são barradas; saem no máximo 4, ordenadas	Não (puro Python)
C — Time-travel	A edição de checkpoint muda o estado (<code>before ≠ after</code>) e cria um ramo novo	Sim

(Histórico da entrega: o código foi implementado e validado com LLM simulado quando a chave da API estava sem créditos; os casos A e C foram posteriormente re-executados com a API real — os três passam — e as tags `stop-1`, `stop-2` e `stop-3` foram publicadas no repositório.)

O que o módulo 3 entregou, em uma frase

Uma equipe de agentes especialistas — cada um um grafo independente com suas ferramentas selecionadas dinamicamente sob política de segurança — coordenada por um supervisor, com aprovação humana embutida nas ações sensíveis e a capacidade de auditar e **consertar execuções passadas** sem refazê-las.

Passo a passo: como o código foi construído

O stop 3 preencheu 13 blocos `TODO` em quatro arquivos — e corrigiu um bug do scaffolding no caminho. A ordem de implementação seguiu as dependências: primeiro o seletor (não depende de nada), depois os especialistas (usam o seletor indireto via catálogo), depois o supervisor (usa os especialistas), por fim o time-travel (usa o grafo pronto).

Passo 0 — O bug do scaffolding (`_resolve_tools`)

Antes de qualquer TODO, o grafo nem compilava. O catálogo fornecido (Task 0) define cada ferramenta como uma função anônima (*lambda*) — que não tem nome nem docstring próprios. E tanto o executor (`ToolNode`) quanto o `bind_tools` do LLM **precisam de uma descrição** para montar o esquema da ferramenta. A correção:

```
def _resolve_tools(allow_list):
    """As entradas do CATALOG são lambdas cruas sem docstring — a conversão
    automática do LangChain não consegue montar o schema delas. Cada entrada
    é embrulhada explicitamente num StructuredTool usando os campos do
    próprio ToolSpec."""
    return [
        StructuredTool.from_function(
            func=spec.fn,                # a função crua do catálogo
            name=spec.name,              # o nome vem do ToolSpec
            description=spec.use_for,    # a descrição-vitrine vem do ToolSpec
            args_schema=spec.args_schema, # o esquema de argumentos também
        )
        for spec in CATALOG
        if spec.name in allow_list      # só as ferramentas DESTA especialista
    ]
```

A *lição*: o catálogo já tinha toda a informação (nome, descrição, esquema) — só que em campos de dados, não nos lugares onde o framework procura. O embrulho `StructuredTool.from_function` faz a ponte. Diagnosticar isso exigiu ler o erro de compilação do grafo até a causa raiz.

Passo 1 — O BigTool (`selector/bigtool.py`)

Implementação mostrada na seção anterior (filtro → ranking → top-4). O detalhe adicional do passo a passo é o **contexto de seleção** — a "credencial" que o chamador apresenta:

```
class SelectionContext(BaseModel):
    region: str                # onde o pedido está sendo atendido (ex.: "us")
    scopes: set[str]           # permissões de quem pede (ex.: {"orders:read"})
    risk_ceiling: str = "med"  # o risco máximo tolerado neste contexto
```

Cada pedido chega com sua credencial, e o seletor cruza credencial × catálogo. A mesma pergunta pode receber ferramentas diferentes dependendo de **quem** pergunta e **em que contexto** — controle de acesso dinâmico, não fixo.

Passo 2 — Os especialistas como subgrafos (`graph/specialists.py`)

Cada `build_*_specialist()` monta e **compila** um grafo completo. O de pedidos, anotado:

```
def build_order_specialist():
    order_tool_fns = _resolve_tools(ORDER_ALLOW_LIST)    # só ferramentas de pedido
    tool_node = ToolNode(order_tool_fns)

    def _route_after_order_llm(state):
        last = state["messages"][-1]
        if getattr(last, "tool_calls", None):
            return "order_tools"    # pediu ferramenta → executa
        return END    # respondeu → fim DESTESUBGRAFO

    builder = StateGraph(SpecialistState)
    builder.add_node("order_llm", _order_node)    # o cérebro do especialista
    builder.add_node("order_tools", tool_node)    # as mãos
    builder.add_edge(START, "order_llm")
    builder.add_conditional_edges("order_llm", _route_after_order_llm,
                                  {"order_tools": "order_tools", END: END})
    builder.add_edge("order_tools", "order_llm")    # o ciclo pensar→agir

    return builder.compile()    # ← COMPILA aqui: nasce uma peça fechada e plugável
```

É a anatomia da semana 7 em miniatura — cada especialista é um mini-agente completo. A restrição do LangGraph v1 documentada no código: **compile o grafo interno primeiro, depois** `add_node` — a ordem inversa não funciona.

Passo 3 — O portão HITL cirúrgico (dentro do especialista de garantia)

O roteamento do especialista de garantia tem **três saídas** em vez de duas — e é aí que mora a segurança:

```
def _route_after_warranty_llm(state):
    last = state["messages"][-1]
    calls = getattr(last, "tool_calls", None) or []
    if any(call.get("name") == "ticket_create" for call in calls):
        return "warranty_hitl_gate"    # ferramenta de ALTO RISCO → portão primeiro
    if calls:
        return "warranty_tools"    # ferramenta comum → executa direto
    return END    # sem ferramenta → resposta final
```

E o portão em si é um nó de uma linha útil:

```
def _warranty_hitl_gate_node(state):
    # interrupt() SUSPENDE o grafo inteiro aqui. O estado fica no checkpoint.
    # O operador inspeciona a chamada pendente e retoma com Command(resume=True).
    interrupt("ticket_create requires operator approval. Resume to proceed.")
    return {}    # aprovação é binária – retomou, passou; nenhum dado extra necessário
```

A topologia é a política de segurança: consultas de garantia (`warranty_check`, `refund_status` — risco baixo) fluem sem fricção; **só** o ramo do `ticket_create` passa pelo portão. E depois da aprovação,

warranty_hitl_gate → warranty_tools — a ferramenta roda normalmente. No notebook, a retomada é:

```
from langgraph.types import Command
app.invoke(Command(resume=True), config) # o "carimbo do gerente"
```

Passo 4 — O supervisor (graph/supervisor.py)

Com os subgrafos prontos, o supervisor é quase montagem: classificar (supervisor_node , heurística de palavras-chave — order/warranty/kb com kb como fallback), encaminhar (_route_from_supervisor , que valida a rota e usa kb para qualquer valor estranho), plugar os três subgrafos compilados, agregar (aggregator_node extrai a última mensagem do especialista como final_answer).

O contrato de montagem: build_supervisor_graph(checkpointer) **recebe o checkpointer de fora** — os testes passam InMemorySaver (rápido, descartável), um notebook de produção pode passar SqliteSaver (persistente em arquivo). Mesma peça, persistência à escolha do dono.

Passo 5 — O time-travel (timetravel/repair.py)

A implementação (mostrada na seção anterior) segue os cinco passos do docstring: listar histórico → capturar o "antes" → update_state(target_cfg, edit, as_node=...) (que **cria o ramo novo**) → invoke(None, resumed_cfg) (retomar do checkpoint remendado) → devolver o diff {before, after, branch_id}.

O detalhe do as_node: a edição precisa declarar **de qual nó** ela finge vir — porque o LangGraph precisa saber de onde continuar o fluxo. Editar o estado "como se fosse o nó X" faz a retomada seguir as arestas que saem de X.

Passo 6 — A validação e o fechamento

Os três casos de tests/test_stop3.py fecham o ciclo:

- **Caso A** monta o supervisor real e manda uma conversa multi-domínio, afirmando que ≥ 2 especialistas foram acionados (leitura dos checkpoints prova quais subgrafos rodaram).
- **Caso B** ataca o seletor puro: injeta um catálogo com ferramentas doentes, de região errada, sem escopo e de risco alto, e afirma que **nenhuma** passa — e que o resultado tem no máximo 4, ordenados.
- **Caso C** roda o grafo, executa um repair_run com uma edição, e afirma before != after e branch_id não-vazio.

O commit final do stop 3 registra tudo: os 13 TODOs preenchidos, o bug do _resolve_tools corrigido, o caso B passando em qualquer ambiente e os casos A e C validados — primeiro com dublê (quando a chave estava sem créditos), depois re-executados com a API real. As tags stop-1, stop-2 e stop-3 publicadas encerram formalmente a entrega.

Conclusão — A Jornada Completa

A linha do tempo da evolução

Vale recuar e olhar o caminho inteiro. Cada semana respondeu a uma pergunta que a anterior deixou aberta:

Semana	Pergunta que respondeu	O que nasceu
1	"Dá para conversar com um LLM e atender clientes?"	O atendente básico: analisar, responder, resumir, arquivar
2	"Como tornar isso confiável e inspecionável?"	Linha de montagem LCEL, fichas validadas (Pydantic), rastreamento
3	"Como lembrar de tudo e <i>fazer</i> coisas?"	Memória híbrida, 6 ferramentas, agente ReAct, follow-ups no Notion
M1	"Como colocar no ar para pessoas de verdade?"	Site (Next.js) + API (FastAPI) + banco (PostgreSQL) na nuvem
4	"E quando o conhecimento não cabe no prompt?"	RAG: a biblioteca que busca por significado (ChromaDB)
5	"A busca está <i>boa</i> ? Prove."	MMR + re-ranking + experimento de chunk + métricas P@k/MRR
6	"E se o LLM inventar? E tabelas? E imagens?"	Graph RAG, guardrails com verificação frase a frase, multimodal
7	"O que há dentro da caixa-preta do agente?"	StateGraph à mão: estado tipado, trava de loop, checkpoints
8	"Como organizar vários domínios?"	Roteador → especialistas com allow-lists → agregador com citações
9	"Como fazer isso virar um sistema de produção?"	Supervisor + subgrafos + BigTool + aprovação humana + time-travel

Os cinco princípios que atravessam o projeto inteiro

Se as 200 páginas anteriores tivessem que virar cinco frases, seriam estas:

- Não confie, valide.** Do JSON da semana 1 ao verificador de alucinação da semana 6: toda saída de LLM passa por um fiscal antes de seguir adiante.
- Não use IA onde uma regra resolve.** O resumo determinístico (semana 2), o roteador por palavras-chave (semana 8), o filtro de política (semana 9): as partes previsíveis do sistema são

código comum — mais barato, mais rápido, mais auditável.

3. **Falhe com elegância.** Ferramentas que devolvem erros educados em vez de explodir, travas de loop com mensagens honestas, portões que preferem "não sei" a inventar. Sistemas de produção são definidos pelo comportamento nos piores dias.
 4. **Meça antes de melhorar.** As métricas da semana 5 (P@k, MRR), o comparativo baseline × otimizado da semana 6, os testes de aceitação de toda semana: nenhuma melhoria foi declarada — todas foram demonstradas.
 5. **Humano no circuito onde importa.** Escalonamento para time prioritário (semana 1), aprovação antes de criar chamado (semana 9): autonomia é um dial, não um interruptor — e as ações sensíveis sempre têm um humano com a chave.
-

Glossário

Termos em ordem alfabética, explicados sem jargão.

Agente — Um programa de IA que decide sozinho os próprios passos: quais ferramentas usar, em que ordem, quando parar. Diferente de um fluxo fixo, onde o programador decidiu tudo de antemão.

API (Application Programming Interface) — O "balcão de atendimento" que um programa oferece a outros programas: um endereço na internet que recebe pedidos padronizados e devolve respostas padronizadas.

API key (chave de API) — A senha que identifica quem está usando uma API paga. Como um cartão corporativo: a conta chega para o dono.

Alucinação — Quando um LLM responde algo falso com total confiança. O principal risco de chatbots corporativos; combatido com RAG, citações e verificadores.

BigTool — Padrão que, em vez de dar todas as ferramentas ao LLM de uma vez, seleciona dinamicamente as poucas mais relevantes (e permitidas) para cada pedido.

Backend — A "cozinha" de um sistema web: onde a lógica de verdade roda. O usuário nunca vê; o frontend conversa com ele.

Checkpoint — Fotografia do estado de um fluxo num dado momento, gravada automaticamente. Permite retomar, pausar para aprovação e "viajar no tempo".

ChromaDB — Banco de dados vetorial usado no projeto: guarda textos com seus "endereço de significado" e busca pelos mais próximos de uma pergunta.

Chunk — Pedaco de documento (aqui, ~500-700 caracteres) usado como unidade de busca no RAG.

Cross-encoder — Modelo que lê pergunta e trecho *juntos* e dá uma nota de relevância precisa. Mais lento que a busca vetorial; usado para reordenar poucos finalistas.

Deploy — Publicar um sistema em servidores na internet, tornando-o acessível ao público.

Embedding — A conversão de um texto numa lista de números que representa seu *significado*. Textos de significado parecido ganham números próximos — a base da busca semântica.

Endpoint — Um "guichê" específico de uma API: um endereço + uma operação (ex.: `POST /api/chat/sessions` = "abrir sessão de conversa").

FastAPI — Framework Python usado para construir o backend do projeto.

Framework — Biblioteca grande que dita a estrutura do programa; o desenvolvedor preenche as lacunas.

Frontend — O "salão" do sistema: a interface que o usuário vê e toca (site, aplicativo).

Graph RAG — Extensão do RAG que extrai fatos (sujeito → relação → objeto) dos documentos e os liga num grafo, permitindo responder perguntas que cruzam vários documentos.

Guardrail — Trava de segurança em torno de um LLM: verificação de citações, portões de decisão, limites de comportamento.

HITL (Human-In-The-Loop) — "Humano no circuito": o sistema pausa e espera aprovação humana antes de ações sensíveis.

JSON — Formato de texto para organizar dados (a "ficha padronizada"), legível por humanos e programas.

LangChain / LCEL — Biblioteca que organiza chamadas de LLM como linhas de montagem componíveis; LCEL é sua notação (o `|` de "e depois passa para").

LangGraph — Biblioteca para montar fluxos de agentes como grafos: nós (etapas), arestas (caminhos), estado compartilhado, checkpoints.

LangSmith — Plataforma de observabilidade: grava cada execução da linha de montagem com tempos, custos e conteúdos por etapa.

LLM (Large Language Model) — O "cérebro": modelo de IA treinado em textos massivos, capaz de entender e gerar linguagem. No projeto: GPT-4o-mini e GPT-4.1-mini.

MCP (Model Context Protocol) — Padrão aberto ("USB das ferramentas de IA") que permite a qualquer assistente compatível usar ferramentas expostas por um servidor.

MMR (Maximal Marginal Relevance) — Critério de busca que equilibra relevância com diversidade: traz trechos pertinentes *e diferentes entre si*.

MRR (Mean Reciprocal Rank) — Métrica de busca: em que posição veio o primeiro resultado correto? ($1^{\circ} = 1,0$; $2^{\circ} = 0,5...$).

Notebook (Jupyter) — Arquivo interativo que mistura texto, código executável e resultados — o "caderno de laboratório" do projeto.

Pipeline — Sequência de etapas de processamento; sinônimo de "linha de montagem" neste documento.

Precision@k — Métrica de busca: dos k resultados retornados, que fração era realmente relevante?

Prompt — O texto de instruções enviado ao LLM. "Prompt de sistema" = as instruções permanentes (persona, regras).

Pydantic — Biblioteca-fiscal: define moldes de dados e rejeita qualquer coisa fora do formato.

Python — A linguagem de programação do projeto.

RAG (Retrieval-Augmented Generation) — "Prova com consulta": buscar os trechos relevantes de uma base de conhecimento e fazer o LLM responder só com base neles, citando fontes.

ReAct (Reason + Act) — Padrão de agente: pensar → agir (usar ferramenta) → observar o resultado → pensar de novo... até ter a resposta.

Reducer — Regra de acúmulo de um campo do estado no LangGraph (ex.: listas concatenam, contadores somam) — impede que uma etapa apague o trabalho de outra.

Render / Vercel — Provedores de nuvem usados no deploy: Render hospeda backend + banco; Vercel hospeda o frontend.

StateGraph — O fluxograma executável do LangGraph: nós, arestas condicionais e estado tipado.

Temperatura — Dial de criatividade do LLM: 0 = máxima previsibilidade (classificação); valores maiores = mais variação (redação).

Token — A unidade de texto que o LLM processa ($\approx \frac{3}{4}$ de uma palavra) e a unidade de cobrança das APIs.

Tool (ferramenta) — Função do sistema que o LLM pode pedir para executar (consultar pedido, abrir chamado...).

Time-travel — Capacidade de voltar a um checkpoint passado, editá-lo e retomar a execução dali, criando um ramo novo (o original fica preservado para auditoria).

Vector store (banco vetorial) — A "estante mágica": banco de dados que organiza textos pelos seus embeddings e busca por proximidade de significado.

Apêndice — Mapa de repositórios e arquivos

Repositório principal: `techstore-chatbot`

Caminho	Semana	Conteúdo
<code>TechStorePlus_Customer_Service_Chatbot_Project.ipynb</code>	1	Chatbot OpenAI direto
<code>TechStorePlus_LangChain_LCEL_Chatbot.ipynb</code>	2	Refatoração LCEL + Pydantic + LangSmith
<code>src/chains/memory_agent.py</code>	3	Agente ReAct com memória e ferramentas
<code>src/components/hybrid_memory.py</code>	3	Memória híbrida (buffer + resumo + ficha)
<code>src/components/customer_tools.py</code>	3	As 6 ferramentas de atendimento
<code>src/mcp/notion_followup_server.py</code>	3	Servidor MCP de follow-ups
<code>Week4_RAG_TechStore.ipynb</code> + <code>src/rag/</code>	4	Pipeline RAG fundamentos
<code>Week4_RAG_Python_Library.ipynb</code>	4	Mini-projeto RAG (docs da Requests)
<code>Week5_RAG_Optimization.ipynb</code>	5	MMR, re-ranking, experimentos, métricas
<code>Week6_Stop3_Production_RAG.ipynb</code>	6	Graph RAG, guardrails, multimodal
<code>Week7_LangGraph_Challenge.ipynb</code> + <code>src/chains/langgraph_challenge_agent.py</code>	7	StateGraph à mão
<code>tests/</code>	—	Testes automatizados de todas as fases

Repositórios de desafio

Repositório	Módulo	Conteúdo
c03-t05-bruno-pieri-m1-challenge	M1	Backend FastAPI + frontend Next.js + deploy Render/Vercel
c03-t05-bruno-pieri-m2-challenge	M2	Pipeline RAG completo (loader, vectorstore, reranker, guardrails, graph, multimodal, métricas)
c03-t05-bruno-pieri-m3-challenge	M3	Stops 1-3: agente básico → roteador/especialistas → supervisor + BigTool + HITL + time-travel

Documento gerado a partir do código-fonte real dos repositórios do projeto TechStore Plus — semanas 1 a 9 do programa DevOps/ML Serving da Pluralit. Todos os trechos de código citados são extratos fiéis dos arquivos indicados em cada capítulo.